

Coding Interview Preparation

Maulana Hafidz Ismail

22 Nov 2025

Contents

1	Pengenalan ke Interview Coding	1
1.1	Proses-proses Interview	1
1.1.1	Proses Pengajuan Lamaran	1
1.1.2	Proses Wawancara	1
1.1.3	Proses Kalibrasi	1
1.2	Persiapan untuk Sukses	1
1.3	Langkah-langkah dalam Wawancara Teknis	1
1.4	Menggunakan Alat yang Tepat	2
1.5	Optimasi Kode	2
1.6	Tipe-tipe interview	2
1.6.1	Screening	2
1.6.2	Quiz	3
1.6.3	Online Coding Assignment	3
1.6.4	The Technical Interview	4
1.6.5	The take-home assignment	4
1.6.6	Top Tips	5
1.7	Kommunikasi Non-Verbal	5
1.8	Komunikasi Verbal	5
1.9	Jenis Wawancara	5
1.10	Persiapan untuk Wawancara	6
1.11	Kualitas yang Dicari	6
1.12	Pseudocode	6
1.12.1	Kenapa menggunakan Pseudocode	6
1.12.2	Kapan harus menulis Pseudocode	6
1.12.3	Bagaimana cara menulis Pseudocode	7
1.13	Tips Interview	8
1.13.1	Persiapan	8
1.13.2	Soft skills	9
1.13.3	Presentasi	9
1.13.4	Demeanor	10
1.13.5	Interview Virtual	10
1.14	Melakukan Tes terhadap Solusi	10
1.14.1	Tugas Take-home	11
1.14.2	Interview Coding	11
1.15	Angka Biner	13
1.16	Representasi dan Penyimpanan	13
1.17	Contoh Aplikasi Biner dalam Komputer	14
1.18	Contoh Hitung Kombinasi	14
1.19	Bekerja dengan Binary	14
1.19.1	Logika Boolean	14
1.19.2	Tabel Kebenaran	16
1.19.3	Gerbang-gerbang	16
1.20	CPU dan Memori	18
1.21	Jenis-jenis Memori	18

1.22	Pendekatan untuk Membuat Solusi di Komputer Sains	18
1.22.1	Mengidentifikasi masalah	18
1.22.2	Merumuskan model	19
1.22.3	Menyempurnakan solusi	20
1.23	Evaluasi Time Complexity	22
1.24	Contoh Kompleksitas Waktu	22
1.25	Kasus Terbaik, Terburuk, dan Tengah	22
1.26	Pertanyaan untuk Evaluasi	23
1.27	Bekerja dengan Kompleksitas Waktu	23
1.27.1	Pendahuluan	23
1.27.2	Mana Pengukuran yang Benar Merefleksikan Kemungkinan Eksekusi yang Paling Cepat dari Sebuah Kode?	23
1.27.3	Representasi Visual dari Sebuah Masalah	28
1.27.4	Kasus buruk, Kasus baik, dan Kasus rata-rata	28
1.28	Konsep Dasar	29
1.29	Jenis Ruang dalam Kompleksitas	29
1.30	Contoh Perhitungan	29
1.31	Pertimbangan Desain	29
2	Pengenalan ke Struktur Data	30
2.1	Pengantar Struktur Data	30
2.2	Jenis Struktur Data	30
2.3	Pertimbangan dalam Memilih Struktur Data	30
2.4	Strings	31
2.4.1	Representasi String	31
2.4.2	Kegunaan Umum	32
2.4.3	Imutabilitas	32
2.5	Integers	33
2.5.1	Representasi Integer	33
2.5.2	Alokasi Memori	34
2.5.3	Class Pembungkus	34
2.5.4	Kesimpulan	35
2.6	Booleans	35
2.6.1	Pernyataan Kondisional	35
2.6.2	Operator Logika	37
2.7	Arrays	39
2.7.1	Menginisialisasi Arrays	39
2.7.2	Mengelola Array di dalam Memori	40
2.7.3	Menggabungkan Dua Array	41
2.8	Objects	42
2.8.1	Definisi	42
2.8.2	Contoh	42
2.9	Lists	44
2.10	Sets	44
2.11	List dan Set dalam Bahasa Pemrograman yang Berbeda	45
2.11.1	Mutabilitas	45

2.11.2	Kenapa Set Cepat?	45
2.11.3	List	46
2.12	Stack	46
2.12.1	Contoh Penggunaan Stack	47
2.13	Queue	47
2.13.1	Contoh Penggunaan Queue	47
2.14	Stack dan Queue dalam Bahasa Pemrograman yang Berbeda	47
2.14.1	Stacks	48
2.14.2	Queues	48
2.14.3	Kesimpulan	49
2.15	Struktur Umum Trees	49
2.16	Terminologi Penting pada Trees	49
2.17	Jenis-jenis Trees	49
2.18	Keuntungan Menggunakan Trees	49
2.19	Metode Traversal	50
2.20	Trees dalam Bahasa Pemrograman yang Berbeda	50
2.20.1	Implementasi Tree	50
2.20.2	Searching pada Tree	51
2.20.3	Kesimpulan	51
2.21	Apa itu Hash Table?	51
2.22	Cara Kerja Hash Table	52
2.23	Koalisi dalam Hash Table	52
2.24	Keuntungan Menggunakan Hash Table	52
2.25	Hash Table pada Bahasa Pemrograman yang Berbeda	52
2.25.1	Apa itu HashTable?	52
2.25.2	Implementasi Hashtable pada Kotlin	53
2.25.3	Apa itu thread-safe, dan bagaimana aplikasinya ke hashtable?	53
2.25.4	Bagaimana thread-safe berkaitan dengan implementasi Kotlin untuk Hashtable	53
2.25.5	Implementasi Hashtable pada Python	54
2.26	Apa itu Heap?	54
2.27	Operasi Dasar Heap	54
2.28	Implementasi dan Penggunaan Heap	54
2.29	Aplikasi Heap	55
2.30	Ringkasan Istilah terkait Heap	55
2.31	Struktur Data Graph	55
2.32	Terminologi Graph	55
2.33	Metode Traversal pada Graph	56
2.34	Aplikasi Graph	56
2.35	Heap dan Graph dalam Bahasa Pemrograman yang Berbeda	56
2.35.1	Terminologi	56
2.35.2	NetworkX	56
2.35.3	Heaps	57
2.35.4	Kesimpulan	57

3	Pengenalan ke Algoritma	58
3.1	Algoritma Sorting	58
3.2	Selection Sort	58
3.3	Insertion Sort	58
3.4	Quicksort	59
3.5	Struktur Data Terkait	59
3.6	Time Complexity dan Space Complexity pada Algoritma Sorting	59
3.6.1	Selection Sort	59
3.6.2	Quicksort	61
3.6.3	Kesimpulan	63
3.7	Linear Searching	63
3.8	Binary Searching	64
3.9	Pertimbangan dalam Searching	64
3.10	Time Complexity dan Space Complexity dalam Algoritma Searching	64
3.10.1	Linear Search	64
3.10.2	Binary Search	65
3.10.3	Kesimpulan	67
3.11	Prinsip Dasar Paradigma Divide and Conquer	67
3.11.1	Contoh: Merge Sort	67
3.12	Keuntungan Paradigma Divide and Conquer	68
3.13	Apa itu Rekursi?	68
3.14	Tiga Persyaratan untuk Solusi Rekursif	68
3.15	Contoh Penggunaan Rekursi	69
3.16	Keuntungan Menggunakan Rekursi	69
3.17	Dynamic Programming	69
3.18	Memoization	69
3.19	Recursion	69
3.20	Contoh Masalah terkait Dynamic Programming	69
3.21	Langkah-langkah dalam Dynamic Programming	70
3.21.1	Sintaks Contoh	70
3.22	Prinsip Dasar Greedy Algorithm	70
3.23	Perbandingan Greedy Algorithm dengan Dynamic Programming	70
3.24	Contoh Penerapan Greedy Algorithm dan Dynamic Programming	70
3.25	Contoh Jalur Terpendek	71
3.26	Kesimpulan terkait Greedy Algorithm	71

1 Pengenalan ke Interview Coding

1.1 Proses-proses Interview

1.1.1 Proses Pengajuan Lamaran

- Resume Screening: Proses di mana perekrut menilai resume dan pengalaman kerja kandidat untuk menentukan kesesuaian dengan posisi yang dilamar.
- Interview with Recruiter: Pertemuan antara kandidat dan perekrut untuk mendiskusikan keterampilan dan pengalaman, serta memastikan kecocokan untuk peran.

1.1.2 Proses Wawancara

- Technical Interviews: Wawancara yang berfokus pada kemampuan teknis kandidat, termasuk coding challenges dan pertanyaan tentang algoritma dan data structures.
- Behavioral Interviews: Wawancara yang mengeksplorasi bagaimana kandidat berkolaborasi dan menyelesaikan masalah dalam situasi kerja.

1.1.3 Proses Kalibrasi

- Discussion of Candidate Performance: Tim yang terlibat dalam wawancara berkumpul untuk mendiskusikan kinerja kandidat dan memutuskan apakah akan memberikan tawaran pekerjaan.
- Boot Camp Process: Proses pelatihan bagi kandidat yang diterima untuk mempelajari cara kerja di perusahaan dan memilih tim yang diinginkan.

1.2 Persiapan untuk Sukses

- Technical Interview: Wawancara di mana Anda menunjukkan kemampuan teknis dan soft skills yang sesuai dengan perusahaan.
- Soft Skills: Kemampuan sosial seperti komunikasi yang jelas dan etika kerja yang baik.

1.3 Langkah-langkah dalam Wawancara Teknis

- Solve the Problem Conceptually First: Sebelum menulis kode, pastikan Anda memahami pertanyaan dengan jelas. Gunakan whiteboard untuk mencatat poin-poin penting.
- Pseudocode: Menulis langkah-langkah solusi dalam bentuk teks yang menyerupai kode, membantu Anda merumuskan ide sebelum menulis kode sebenarnya.

1.4 Menggunakan Alat yang Tepat

- Data Structure: Struktur yang digunakan untuk menyimpan dan mengorganisir data. Contoh: menggunakan dictionary untuk menghitung jumlah pasang kaus kaki berdasarkan warna.
- Algorithm: Langkah-langkah sistematis untuk menyelesaikan masalah. Penting untuk memahami algoritma dan cara visualisasinya.

1.5 Optimasi Kode

- Time Complexity: Ukuran seberapa cepat solusi Anda dapat dijalankan.
- Space Complexity: Ukuran seberapa banyak memori yang digunakan oleh solusi Anda.
- DRY Principle (Don't Repeat Yourself): Prinsip untuk menghindari pengulangan kode, sehingga kode lebih efisien dan mudah dipelihara.

1.6 Tipe-tipe interview

1.6.1 Screening

Kontak pertama Anda dengan sebuah perusahaan akan berupa wawancara penyaringan. Seorang perekrut, setelah membaca lamaran Anda, biasanya akan mengatur panggilan telepon untuk membahas kesesuaian potensial Anda. Tahap ini bertujuan untuk menentukan apakah Anda benar-benar tertarik dengan posisi tersebut dan apakah Anda cocok dengan perusahaan. Ini adalah kesempatan yang baik bagi Anda, sebagai calon karyawan, untuk mempelajari lebih lanjut tentang posisi, perusahaan, dan proses perekrutan mereka. Beberapa pertanyaan yang baik untuk diajukan antara lain:

- Apa peran yang dimaksud?
- Bahasa pemrograman apa yang biasanya digunakan oleh perusahaan?
- Bagaimana proses wawancara, jenis wawancara apa yang akan dilakukan, dan berapa banyak wawancara yang akan ada?
- Apa sifat proyek yang akan ditangani oleh peran yang diusulkan?
- Bagaimana komposisi tim yang biasanya ada?
- Siapa saja yang akan menjadi anggota panel wawancara, dan apa peran mereka di perusahaan?

Ingatlah bahwa pewawancara ingin memberikan Anda kesempatan untuk menyampaikan argumen yang meyakinkan mengapa Anda layak untuk dipekerjakan. Mereka seharusnya bersedia memberikan informasi apa pun yang dapat membantu Anda mempersiapkan diri dengan lebih baik.

Dari sudut pandang perusahaan, proses seleksi biasanya akan fokus pada keterampilan lunak atau interpersonal Anda. Tidak akan ada pertanyaan teknis, dan proses ini biasanya akan dilakukan oleh perwakilan Sumber Daya Manusia (SDM). Menampilkan diri dengan baik dan menunjukkan kemampuan sosial Anda, seperti mendengarkan dan menjawab pertanyaan, sangat penting.

1.6.2 Quiz

Ujian kuis adalah cara yang sangat sederhana untuk menentukan apakah seorang kandidat mampu mengkodekan. Biasanya, ujian ini dilakukan pada tahap awal proses wawancara dan berfungsi sebagai penyaring untuk mengurangi jumlah wawancara yang harus dilakukan oleh perusahaan. Anda tidak boleh mengharapkan pertanyaan yang mendalam atau panjang; tujuan utamanya adalah untuk menentukan apakah kandidat memiliki pemahaman yang baik tentang dasar-dasar pemrograman. Beberapa pertanyaan yang mungkin diajukan antara lain:

- Bagaimana cara memeriksa nilai null dalam array menggunakan bahasa pemrograman pilihan Anda?
- Apa kompleksitas ruang untuk algoritma Quicksort?
- Struktur data apa yang akan Anda gunakan untuk menyimpan daftar kunci dan nilai?
- Apa yang dimaksud dengan tabrakan (collision) dalam hashing?
- Apa sintaksis untuk memilih nama kolom menggunakan SQL?
- Proses pengujian apa yang Anda kenal?

Pertanyaan-pertanyaan tersebut akan bersifat spesifik untuk posisi yang dilamar, sehingga isi pertanyaannya dapat bervariasi. Namun, pertanyaan-pertanyaan tersebut harus dijawab dengan jawaban yang ringkas dan langsung. Saat mempersiapkan diri untuk wawancara, disarankan untuk mengidentifikasi bahasa pemrograman apa yang digunakan oleh perusahaan dan informasi teknis lainnya yang dapat diperoleh dari deskripsi pekerjaan dan proses seleksi awal.

1.6.3 Online Coding Assignment

Anda mungkin diminta untuk menyelesaikan tugas pemrograman online, tergantung pada posisi yang Anda lamar. Hal ini umumnya dilakukan sebelum wawancara teknis, karena hasilnya dapat digunakan sebagai bahan pembahasan selama wawancara selanjutnya. Namun, tugas tersebut juga dapat dilakukan setelah wawancara teknis yang sukses untuk membandingkan kandidat.

Langkah ini memungkinkan Anda untuk mendekati dan menyelesaikan tugas yang jelas didefinisikan dalam lingkungan yang tidak menekan. Mirip dengan tingkat kesulitan kuis, ini adalah langkah penyaringan untuk memastikan Anda dapat bekerja dengan teknologi target dan memahami dasar-dasarnya,

seperti loop, Boolean, pengecekan, dan sebagainya. Beberapa tugas pemrograman mungkin dibatasi waktu; dalam situasi ini, Anda akan memiliki akses ke Google dan bahan referensi lainnya.

1.6.4 The Technical Interview

Ini juga disebut sebagai tantangan pemrograman. Biasanya, hal ini akan dilakukan setelah tahap seleksi awal. Tantangan ini dapat dilakukan secara virtual atau tatap muka. Pewawancara akan memberikan Anda masalah yang menantang dan memberi Anda waktu singkat untuk menyelesaikannya. Selalu lebih baik untuk mengutarakan pemikiran Anda secara lisan saat menghadapi tantangan tersebut. Misalnya, sebelum memutuskan antara dua struktur data, jelaskan mengapa Anda merasa salah satunya lebih cocok untuk tugas tersebut. Manfaatkan kesempatan ini untuk menunjukkan pemahaman umum Anda tentang topik dan keahlian khusus yang relevan dengan tugas.

Tingkat kesulitan wawancara teknis seharusnya rendah. Pewawancara menyadari bahwa Anda tidak memiliki akses ke Google dan mungkin harus menghafal beberapa sintaks yang diperlukan. Sebaliknya, wawancara ini dirancang untuk menunjukkan kemampuan pemecahan masalah Anda. Beberapa wawancara teknis mungkin dilakukan menggunakan pseudocode. Selain itu, wawancara teknis memungkinkan rekan kerja masa depan untuk menilai kemampuan Anda dalam berargumentasi dengan kode. Mereka sering kali mencakup prompt singkat dan spesifik seperti:

- Bagaimana Anda menjelaskan teknologi X kepada orang yang tidak memiliki latar belakang teknis?
- Teknologi apa yang paling Anda sukai dan mengapa?
- Database apa saja yang pernah Anda gunakan?
- Ceritakan tentang tantangan teknis yang pernah Anda atasi dalam sebuah proyek.
- Proyek apa saja yang pernah Anda kerjakan di waktu luang Anda?

Ini adalah pertanyaan umum, dan wawancara teknis akan disesuaikan dengan peran spesifik. Namun, menyiapkan beberapa jawaban yang menggambarkan perjalanan Anda sebagai programmer adalah praktik yang baik. Tinjau proyek-proyek Anda dan jelaskan dengan jelas berbagai keputusan yang telah Anda ambil dan alasannya, terutama saat memilih satu teknologi daripada yang lain.

1.6.5 The take-home assignment

Jika Anda sampai sejauh ini, Anda sudah melakukan dengan sangat baik. Hal ini mungkin terjadi sebelum atau setelah wawancara teknis, karena pewawancara mungkin ingin mendiskusikan proses berpikir Anda terkait solusi yang

Anda berikan sebelumnya dalam proses ini. Jika ada beberapa putaran wawancara untuk posisi ini, maka hal ini mungkin terjadi di antara dua wawancara teknis. Sebuah perusahaan tidak boleh meminta Anda untuk mengerjakan tugas di rumah kecuali mereka merasa Anda berpotensi cocok untuk posisi tersebut.

Anda akan diberikan beberapa hari untuk mengembangkan solusi untuk tantangan tersebut. Anda mungkin diminta untuk membangun aplikasi atau memecahkan masalah rute lalu lintas secara programatik. Tugas lain mungkin spesifik proyek, misalnya, jika Anda akan bekerja dengan pemrosesan gambar, Anda mungkin diminta untuk menunjukkan bahwa Anda dapat mendeteksi objek secara otomatis dalam aliran video.

1.6.6 Top Tips

Lakukan riset sebelum setiap wawancara dan tinjau setiap wawancara setelah selesai. Sebagian besar perusahaan senang memberikan umpan balik setelah wawancara Anda, jadi selalu mintalah hal ini dan gunakan poin-poin yang disampaikan sebagai titik awal untuk mempersiapkan diri dengan lebih baik untuk wawancara berikutnya. Ingat, latihan membuat sempurna.

1.7 Komunikasi Non-Verbal

- **Punctuality:** Datang tepat waktu, idealnya 10 menit sebelum wawancara, menunjukkan keseriusan dan kesiapan.
- **Eye Contact:** Mempertahankan kontak mata selama wawancara untuk menunjukkan perhatian dan keterlibatan.

1.8 Komunikasi Verbal

- **Clear and Concise Language:** Gunakan bahasa yang jelas dan ringkas saat menjawab pertanyaan untuk menghindari kebingungan.
- **STAR Method:** Metode untuk menjawab pertanyaan wawancara dengan menyusun jawaban berdasarkan Situasi, Tugas, Aksi, dan Hasil.

1.9 Jenis Wawancara

- **Technical Interview:** Wawancara di mana kandidat menunjukkan kemampuan teknis mereka melalui pertanyaan coding dan algoritma.
- **Architecture Interview:** Wawancara yang berfokus pada bagaimana membangun fitur end-to-end, termasuk pertimbangan tentang aliran data dan skalabilitas.
- **Behavioral Interview:** Wawancara yang mengeksplorasi pengalaman kandidat dalam bekerja dengan orang lain, tantangan yang dihadapi, dan proyek menarik yang telah dikerjakan.

1.10 Persiapan untuk Wawancara

- Practice: Latihan dengan menjelaskan solusi saat menulis kode, termasuk pertimbangan tentang time complexity dan space complexity.
- Dress Code: Kenakan pakaian yang nyaman; tidak perlu formal seperti jas. T-shirt dan jeans sudah cukup.
- Communication: Penting untuk menjelaskan pemikiran saat coding, terutama jika mengalami kesulitan.

1.11 Kualitas yang Dicari

- Enthusiasm: Kandidat yang menunjukkan semangat dan sikap positif lebih disukai.
- Feedback Reception: Kemampuan untuk menerima umpan balik dan berkolaborasi dalam menyelesaikan masalah adalah nilai tambah.

1.12 Pseudocode

1.12.1 Kenapa menggunakan Pseudocode

Pseudocode adalah langkah awal yang sah saat mulai merancang solusi. Pseudocode adalah representasi tingkat tinggi dari ide-ide yang ditulis dengan cara yang mirip dengan kode. Pada dasarnya, pseudocode dapat menjadi alat bantu untuk menyoroti bagi diri sendiri elemen-elemen apa yang harus termasuk dalam program.

Setiap baris akan memberi Anda kesempatan untuk berhenti sejenak dan mempertimbangkan apa yang diperlukan untuk mencapai hasil yang diinginkan. Saat menulis aplikasi, keputusan yang diambil pada langkah 3 dapat memengaruhi cara langkah 6 harus dikodekan. Setiap tahap memiliki unsur keterbatasan yang memengaruhi cara tahap berikutnya akan diselesaikan.

Pertimbangkan bahwa Anda sedang menulis aplikasi yang harus menyimpan data pada langkah 3. Anda memutuskan untuk menggunakan array sebelum melanjutkan. Pada langkah 6, Anda menyadari bahwa beberapa pencarian diperlukan untuk aplikasi tersebut. Saat menulis pseudocode, mudah untuk kembali ke langkah 3 dan mengubah struktur data menjadi sesuatu yang lebih kompatibel dengan langkah 6, seperti menggunakan kamus (dictionary) daripada array. Perubahan kecil dapat meningkatkan beban kerja jika implementasi sudah dimulai karena hal itu mengubah cara langkah 4 dan 5 beroperasi.

1.12.2 Kapan harus menulis Pseudocode

- Sebagai pemula, saat merencanakan kemajuan yang direncanakan dalam pendekatan Anda.
- Sebagai programmer berpengalaman, saat berusaha mengatasi masalah yang kompleks.

- Jika Anda mencoba menjelaskan konsep kepada audiens yang berpengaruh, seperti tim atau klien potensial.
- Jika Anda memberikan petunjuk tentang pekerjaan Anda untuk programmer masa depan yang mungkin akan memelihara kode atau aplikasi yang Anda tulis.
- Dalam wawancara, saat menunjukkan kemampuan Anda untuk menganalisis dan memecahkan masalah.

1.12.3 Bagaimana cara menulis Pseudocode

Tidak ada satu cara baku untuk menulis pseudocode. Setiap organisasi mungkin memiliki standar tersendiri. Secara umum, dapat dikatakan bahwa pseudocode dapat dianggap sebagai representasi teks apa pun yang menggambarkan operasi suatu program.

Pertimbangkan cara mewakili tantangan FizzBuzz yang digunakan untuk menguji kemampuan kandidat dalam berlogika dalam kode:

Tulis program dalam bahasa pemrograman yang ditentukan yang mengiterasi angka dari 1 hingga 40. Cetak angka untuk setiap angka kecuali kelipatan tiga, dalam hal ini cetak Fizz. Untuk kelipatan lima, cetak Buzz, dan untuk kelipatan 3 dan 5, cetak FizzBuzz.

Anda mungkin mulai dengan mewakili setiap persyaratan sebagai baris pseudocode:

```
FOR number 1 to 40
  IF multiple of 3
    output(Fizz)
  IF multiple of 5
    output(Buzz)
  IF multiple of 3 and 5
    output(FizzBuzz)
  ELSE
    output(number)
```

Figure 1:

Kunci dari tugas ini adalah mengetahui cara mengatur pernyataan kondisional. Baris 1 menunjukkan bahwa akan ada iterasi. Dalam hal ini, indentasi menunjukkan bahwa delapan baris kode berikutnya merupakan bagian dari iterasi ini. Sebagai alternatif, blok berikut dapat ditambahkan:

```
START FOR LOOP

#code

END FOR LOOP
```

Figure 2:

Jelas bahwa ada tiga pernyataan bersyarat dan kemudian klausa else yang mencakup semua kasus. Hasil kode dapat dibayangkan dengan melihat pseudocode. Pernyataan else akan berfungsi dengan baik, hanya mencetak angka yang bukan kelipatan 3 dan 5, tetapi ada masalah dengan cara kode menangani angka 15, yang mencetak contoh Fizz, Buzz, dan FizzBuzz.

```
FOR number 1 to 40
  IF multiple of 3 and 5
    output(FizzBuzz)
  ELSE IF multiple of 3
    output(Fizz)
  ELSE IF multiple of 5
    output(Buzz)
  ELSE
    output(number)
```

Figure 3:

Menganalisis contoh pertama pertanyaan dalam bentuk pseudocode memungkinkan Anda untuk mengidentifikasi di mana masalah mungkin terjadi. Secara visual, jauh lebih mudah untuk melihat bagaimana pernyataan kondisional seharusnya disusun. Pertama, inti dari pertanyaan ini terletak pada urutan penempatan pernyataan kondisional dan penggunaan pernyataan kondisional eksklusif. Dalam contoh di atas, "else if" digunakan sebagai pengganti "if" saja. Kedua, urutan pernyataan memiliki dampak yang penting. Anda dapat mencoba output di atas dengan menjalankan kode yang terdapat di bawah ini.

```
for number in range(1, 41):
    if number % 3 == 0 and number % 5 == 0:
        print("FizzBuzz")
    elif number % 3 == 0:
        print("Fizz")
    elif number % 5 == 0:
        print("buzz")
    else:
        print(number)
```

1.13 Tips Interview

1.13.1 Persiapan

Berikut adalah beberapa pertanyaan standar yang mungkin Anda temui dalam wawancara apa pun:

- Ceritakan tentang diri Anda.
- Mengapa Anda merasa bahwa mereka seharusnya mempekerjakan Anda?
- Apa kelebihan utama Anda?

- Atau, apa kelemahan utama Anda?
- Berapa gaji yang Anda harapkan?
- Bagaimana pengalaman Anda sebelumnya membuat Anda cocok untuk peran ini?
- Apa yang dikatakan teman-teman Anda tentang Anda?
- Mengapa Anda ingin peran ini?
- Bagaimana Anda menangani konflik di masa lalu?

Mengetahui pertanyaan-pertanyaan ini akan datang memberikan keuntungan karena Anda dapat mempersiapkannya. Setelah membaca daftar ini, apakah Anda dapat menulis jawaban yang ringkas untuk masing-masing pertanyaan? Anda akan menemukan jawaban untuk banyak pertanyaan ini dalam resume Anda. Strategi yang baik adalah meninjau resume Anda sebelum wawancara untuk melihat aspek mana yang relevan dengan perusahaan dan posisi yang dilamar.

Jawaban tambahan yang layak dipersiapkan mungkin berkaitan dengan perusahaan itu sendiri. Di mana perusahaan tersebut bermarkas, dan apa yang mereka lakukan? Bagaimana pengalaman kerja Anda sebelumnya berkaitan dengan kebutuhan mereka saat ini? Teliti peran yang sama di perusahaan lain. Ketahui gaji pasar untuk posisi tersebut jika ditanya tentang besaran gaji. Pewawancara sudah tahu jawabannya, jadi mampu menjawab ini dengan baik menunjukkan kesiapan Anda. Mengetahui tentang perusahaan menunjukkan antusiasme untuk bekerja di sana. Akhirnya, pertimbangkan bahwa kelemahan juga dapat menjadi kekuatan. "Saya terlalu fokus pada detail" dapat merugikan jika ada tenggat waktu yang ketat; namun, hal itu menjadi kekuatan jika pekerjaan tersebut membutuhkan perhatian detail yang teliti.

1.13.2 Soft skills

Penyelesaian konflik merupakan keterampilan lunak yang sangat penting untuk dimiliki. Dalam kehidupan kerja Anda, Anda pasti akan menghadapi situasi di mana pendekatan atau tujuan Anda tidak sejalan dengan rekan kerja, atasan, atau pelanggan. Mengelola situasi-situasi ini membantu menciptakan lingkungan kerja yang harmonis. Sadari kapan Anda pernah berselisih dengan orang lain dan bagaimana Anda menyelesaikannya. Apakah pendekatan ini berhasil? Jika tidak, lakukan riset tentang cara menangani konflik di tempat kerja.

1.13.3 Presentasi

Seringkali dikatakan, "Berpakaianlah sesuai dengan pekerjaan yang kamu inginkan, bukan pekerjaan yang kamu miliki." Cara kamu tampil dapat secara tidak sadar menyampaikan banyak informasi tentang dirimu. Berpakaianlah rapi dan sopan. Rekan kerja masa depanmu mungkin ada di panel, dan kamu ingin membuat kesan yang baik. Pakaian yang kamu kenakan harus sesuai dengan

peran yang kamu lamar. Kebanggaan terhadap penampilan dapat disamakan dengan kebanggaan terhadap peran yang diemban.

1.13.4 Demeanor

Tunjukkan kesiapan Anda untuk peran tersebut. Rekan kerja ingin melihat seseorang yang tertarik untuk berada di sana. Tetap tenang dan rileks saat menjawab pertanyaan. Tidak pernah disarankan untuk memotong pertanyaan. Dengarkan hingga pertanyaan selesai, lalu ambil napas. Pertanyaan tentang kompetensi memerlukan jawaban STAR (Situasi, Tugas, Tindakan, Hasil). Tunjukkan kebanggaan atas pengalaman kerja sebelumnya yang Anda miliki. Siapkan beberapa pertanyaan. Dalam wawancara, pewawancara selalu harus memberi Anda kesempatan untuk bertanya. Manfaatkan waktu ini untuk mengetahui apa yang dicari. Apa kondisi yang mungkin lebih menguntungkan bagi Anda? Apakah tujuan perusahaan sejalan dengan tujuan Anda, dan apakah Anda dapat mengkomunikasikannya?

Jadilah autentik dalam tindakan Anda. Lebih baik jujur dan terbuka saat melamar posisi. Berlebihan mungkin mengarah pada wawancara lanjutan, tetapi lebih baik mampu mengisi posisi apa pun yang Anda inginkan.

1.13.5 Interview Virtual

Banyak wawancara dilakukan secara virtual. Wawancara virtual menantang karena lebih sulit membaca bahasa tubuh melalui video. Karena kualitas suara, penting untuk tidak berbicara di atas orang lain. Pastikan semua perangkat teknologi Anda berfungsi dengan baik. Pastikan komputer Anda terhubung ke sumber daya dan memiliki headphone yang berfungsi jika terjadi masalah suara. Pastikan video Anda berfungsi dan kompatibel dengan platform. Terkadang, komputer perlu mengubah izin sebelum Anda dapat menggunakan kamera. Dan jangan lupa untuk menguji kecepatan koneksi.

Seperti halnya wawancara tatap muka, penampilan sangat penting. Luangkan waktu untuk memastikan Anda berpakaian sesuai. Meskipun Anda tidak berada di kantor, berpakaianlah seolah-olah Anda berada di sana. Seperti halnya wawancara tatap muka, perhatikan bahasa tubuh Anda. Duduklah dengan tegak dan tetap fokus. Meskipun Anda berada di rumah, tetaplah berperilaku sesuai etika kerja.

Temukan lokasi yang tenang dan bebas dari gangguan. Memiliki ruang kerja khusus dapat membantu. Hal ini diperlukan untuk memastikan bahwa selama wawancara, tidak ada tamu tak terduga yang melakukan pekerjaan rumah di latar belakang.

1.14 Melakukan Tes terhadap Solusi

Pengujian merupakan bagian penting dalam pengembangan perangkat lunak. Idealnya, cakupan pengujian Anda harus komprehensif; namun, selalu ada faktor eksternal yang dapat memengaruhi hasil ideal ini. Secara historis, pen-

gujian sering dilakukan di akhir proses, sehingga jika tenggat waktu produksi mendekat, pengujian sering menjadi hal pertama yang dikurangi. Test-Driven Development (TDD) adalah pendekatan yang dikembangkan untuk menghindari hasil tersebut. Pembahasan tentang pengujian ini akan berfokus pada dua contoh, yaitu wawancara coding dan tugas yang dibawa pulang.

1.14.1 Tugas Take-home

Jika Anda diizinkan untuk membawa kode pulang, Anda mungkin memiliki waktu untuk mengimplementasikan tes yang lebih komprehensif.

Ada banyak jenis tes yang dapat Anda pilih untuk dijalankan. Misalnya:

- Tes integrasi: Tes ini menguji bagaimana berbagai komponen aplikasi berinteraksi satu sama lain. Tes integrasi tidak akan mendalam karena ketergantungan eksternal akan disimulasikan daripada menggunakan instance aktual. Selain itu, unit testing untuk menilai baris kode yang lebih sepsifik, sementara uji integrasi mengambil pendekatan yang lebih global.
- Uji fungsional: Ini adalah uji otomatis yang membuktikan bahwa sistem beroperasi sesuai harapan. Uji ini berfokus pada kemampuan sistem.
- Uji regresi: Ini untuk memastikan bahwa perubahan tidak menyebabkan kesalahan pada kode yang sudah ada. Uji ini memastikan bahwa ketika perubahan sistem dilakukan, hal itu tidak memengaruhi operasinya.

Tingkat pengujian yang Anda terapkan selalu bergantung pada waktu yang tersedia. Bagian penting dari proses ini adalah mengembangkan aplikasi yang berfungsi. Pengujian menunjukkan ketelitian Anda dan memastikan bahwa aplikasi tersebut akan berfungsi dengan baik. Sejauh mana Anda dapat menerapkan strategi-strategi ini mungkin terbatas tergantung pada waktu yang Anda miliki.

1.14.2 Interview Coding

Unit testing adalah pendekatan yang memastikan kode Anda berfungsi sesuai yang diharapkan. Meskipun banyak pengujian ditulis sebelum kode masuk ke produksi, Anda tidak akan memiliki kesempatan untuk menulis semua pengujian ini saat mengikuti wawancara coding. Pengujian unit adalah pendekatan yang dapat dikelola dan dapat diintegrasikan ke dalam solusi Anda, menunjukkan perhatian Anda terhadap detail. Hal ini akan menunjukkan praktik pengujian kode yang baik.

Pengujian unit kurang fokus pada operasi keseluruhan aplikasi. Sebaliknya, pengujian ini memastikan bahwa setiap komponen individu berfungsi sesuai yang diharapkan.

Pertimbangan ketika menulis test

- Keterbacaan: Buatlah tes yang mudah dibaca oleh pengembang lain. Kebiasaan ini harus diinternalisasi terlepas dari apakah Anda bekerja dalam tim. Tes dengan tujuan yang jelas dapat mengidentifikasi masalah jika terjadi. Mereka juga menunjukkan apa yang seharusnya dicapai oleh bagian kode tertentu. Hal ini juga meningkatkan keterpeliharaan kode.
- Hasil yang jelas: Tes Anda sebaiknya, sejauh mungkin, bersifat deterministik. Artinya, hasilnya harus selalu sama terlepas dari kondisi yang ada. Tes deterministik akan selalu gagal jika kode bermasalah ditulis. Tes yang bergantung pada kombinasi kondisi yang berbeda dapat menyulitkan proses debugging.
- Otomatisasi: Tes harus selalu memiliki potensi untuk diotomatisasi sehingga dapat dijalankan dengan cepat setiap kali perubahan dilakukan pada kode. Ini merupakan landasan utama CI/CD (Continuous Integration/Continuous Development).

Menggunakan unit test kedalam praktek Panduan solusi utama menjelaskan langkah-langkah yang diperlukan untuk membuat permainan menebak angka. Di bawah ini adalah tangkapan layar dari pseudocode yang digunakan untuk mengembangkan solusi yang layak.

```
Generate a question
Take in user input
Compare input with answer
Return a result.
```

Figure 4:

Mengambil contoh ini, bayangkan bahwa hanya tersisa beberapa menit dari waktu yang dialokasikan, apa yang akan Anda fokuskan? Uji unit paling dasar yang dapat ditulis adalah kasus asersi, yang memastikan bahwa apa yang diberikan sesuai dengan yang diharapkan. Perpustakaan untuk ini tersedia di sebagian besar bahasa pemrograman.

```

take in user input
// assert not null
// assert all of type integer
// assert within the range specified

generate number
//assert type integer
// assert within a given range

comparing number
generate a mock instance with a given answer
// assert code checks less than
// assert code checks greater than
// assert code correctly acts when number is correct

```

Figure 5:

Langkah pertama mungkin adalah menulis kasus uji untuk menentukan apakah masukan pengguna valid. Baris 2–4 menjelaskan beberapa tes yang dapat diterapkan untuk memvalidasi masukan pengguna, memastikan bahwa sesuatu telah dimasukkan sebelum pengguna menekan tombol return. Anda harus memastikan bahwa masukan tersebut memiliki tipe yang benar dan tidak akan menyebabkan program crash. Selain itu, pastikan masukan tersebut berada dalam batas yang ditentukan. Selanjutnya, pertimbangkan untuk memeriksa angka acak yang dihasilkan. Apakah jenisnya benar dan apakah berada dalam parameter yang dapat diterima? Akhirnya, akan membantu jika Anda mempertimbangkan untuk menyertakan pemeriksaan yang memastikan hasil yang benar saat pernyataan kondisional diaktifkan.

Proses penulisan uji unit telah distandardisasi. Disarankan untuk berlatih mengimplementasikannya saat Anda menulis kode. Hal ini akan membantu Anda cepat familiar dengan proses tersebut dan dapat menyoroti kesadaran Anda dalam lingkungan di mana Anda menguji kode Anda.

1.15 Angka Biner

- Binary Numbers: Sistem angka yang hanya menggunakan dua digit, yaitu 0 dan 1. Komputer menggunakan angka biner untuk menyimpan dan memproses informasi.
- Positional Notation: Metode di mana posisi angka menentukan nilai. Dalam sistem biner, setiap posisi mewakili pangkat dua (2^n).

1.16 Representasi dan Penyimpanan

- Byte: Unit penyimpanan data yang terdiri dari 8 bits. Setiap bit dapat bernilai 0 atau 1.
- Exponentiation: Proses menghitung pangkat suatu angka. Misalnya, $2^3 = 2 * 2 * 2 = 8$. Ini digunakan untuk menentukan jumlah kombinasi yang dapat diwakili oleh byte.

1.17 Contoh Aplikasi Biner dalam Komputer

- Boolean Values: Nilai yang hanya dapat bernilai true (1) atau false (0). Digunakan dalam logika komputer.
- ASCII (American Standard Code for Information Interchange): Kode yang memetakan angka biner ke karakter. Setiap karakter memiliki representasi biner yang unik.

1.18 Contoh Hitung Kombinasi

- Lock Example: Jika sebuah kunci memiliki 4 digit biner, jumlah kombinasi yang mungkin adalah $2^4 = 16$. Jika 5 digit, maka $2^5 = 32$.
- Total Representations in a Byte: Dengan 8 bits, jumlah kombinasi yang dapat diwakili adalah $2^8 = 256$.

1.19 Bekerja dengan Binary

Anda mungkin berpikir bahwa menggunakan hanya dua masukan terlalu membatasi. Faktanya, hal ini sangat fleksibel dan menawarkan berbagai pilihan yang luas ketika digabungkan dengan aljabar Boolean dan sirkuit.

1.19.1 Logika Boolean

Fungsi Boolean memetakan dua masukan ke sebuah nilai. Masukan-masukan ini dibatasi pada dua keadaan. Keadaan-keadaan ini dapat diartikan sebagai: on/off, true/false, atau 1/0. Untuk mendefinisikan fungsi Boolean, Anda hanya perlu menentukan bagaimana keluaran ditentukan untuk semua masukan yang mungkin. Berikut adalah contoh 4 fungsi:

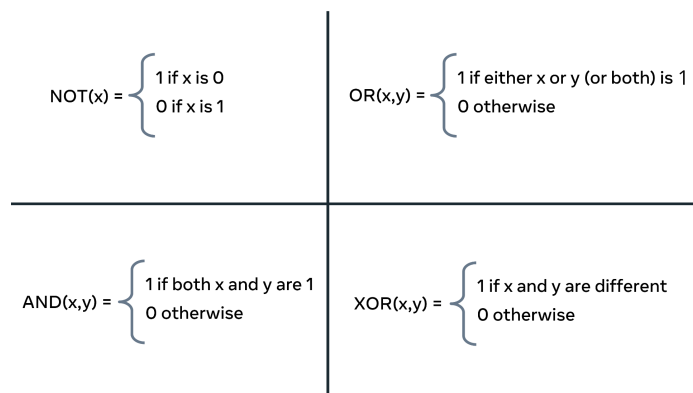


Figure 6:

NOT menerima satu nilai x dan mengembalikan hasilnya sebagai 0 atau 1. OR menerima dua argumen (x , y) untuk menghasilkan keluaran 1. Hanya salah satu dari nilai x atau y yang harus bernilai 1, atau keduanya sekaligus. Jika kedua nilai tersebut 0 secara bersamaan, keluaran adalah 0. AND memerlukan kedua masukan untuk bernilai 1 sebelum dapat menghasilkan 1. Akhirnya, XOR akan menerima dua nilai apa pun dan menentukan apakah keduanya berbeda; jika ya, outputnya adalah 1. Dalam semua kasus lain, hasilnya adalah 0.

NOT	$!x$
AND	$x \ \&\& \ y$
OR	$x \ \ y$
XOR	$x \ ^ \ y$

Figure 7:

Konsep yang sama dapat diwakili secara komputasional. Dalam tabel ini, ekspresi Boolean berada di sebelah kiri, dan simbol komputasionalnya berada di sebelah kanan. Umumnya, ekspresi ini digabungkan dengan pernyataan kondisional atau iterator. Oleh karena itu, Anda dapat mengharapkan kode yang mirip dengan potongan kode berikut:

```
Boolean x = false;
do
{
    System.out.println("executed repeatedly");
}
while(!x);
```

Figure 8:

Dalam kode di atas, sebuah variabel Boolean telah disetel ke false. Sebuah loop do/while kemudian terus-menerus menjalankan kode yang terdapat di dalam blok do hingga nilai x berubah menjadi true. Secara umum, Anda akan mencari hasil tertentu, dan penggunaan logika Boolean memungkinkan Anda untuk terus mencari hingga hasilnya menjadi true. Perhatikan bagaimana fungsi NOT diterapkan dalam loop while untuk menguji apakah iterasi lain harus dilakukan.

1.19.2 Tabel Kebenaran

Mengingat jumlah keluaran yang terbatas dari fungsi Boolean, dimungkinkan untuk menggambarkan semua permutasi ke dalam tabel.

NOT			OR		
x	x'		x	y	x+y
0	1		0	0	0
1	0		0	1	1
			1	0	1
			1	1	1

AND			XOR		
x	y	xy	x	y	$x \oplus y$
0	0	0	0	0	0
0	1	0	0	1	1
1	0	0	1	0	1
1	1	1	1	1	0

Figure 9:

Gambar ini menunjukkan semua kombinasi untuk setiap fungsi Boolean. Kolom X dan Y menunjukkan apakah menerima nilai 0 atau 1, sedangkan kolom XY menunjukkan hasil akhir. Fungsi NOT hanya memiliki satu keluaran dan satu masukan, sehingga tabelnya hanya berukuran 2 x 2. Dibandingkan dengan itu, ketiga contoh lainnya masing-masing menerima dua masukan dan menghasilkan satu hasil.

1.19.3 Gerbang-gerbang

Bahan dasar utama untuk sirkuit logika digital adalah gerbang. Fungsi logika kemudian diimplementasikan melalui interkoneksinya. Gerbang adalah sirkuit elektronik yang menghasilkan keluaran Boolean dari masukan-masukannya.

Name	Graphical Symbol	Algebraic Function	Truth Table															
AND		$F = A \cdot B$ or $F = AB$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	0	1	0	0	1	1	1
A	B	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F = A + B$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	1	1	0	1	1	1	1
A	B	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
NOT		$F = \bar{A}$ or $F = A'$	<table><tr><th>A</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	F	0	1	1	0									
A	F																	
0	1																	
1	0																	
XOR		$F = A \oplus B$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	0	0	1	1	1	0	1	1	1	0
A	B	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																

Figure 10:

Pada gambar di atas, empat fungsi Boolean dasar ditampilkan. Simbol Grafis menunjukkan cara gerbang-gerbang ini digambarkan pada diagram. Anda sering akan menemui diagram sirkuit yang berisi serangkaian gerbang yang saling terhubung. Fungsi Aljabar menunjukkan cara gerbang-gerbang ini dijelaskan ketika ditulis dalam bentuk rumus.

Akhirnya, tabel kebenaran menggambarkan hasil dari masukan tertentu ke gerbang. Pada tingkat dasar, setiap masukan yang dilambangkan dengan A dan B mewakili arus yang dapat mengalir melalui sirkuit. Masukan-masukan ini dapat terhubung ke saklar atau tombol yang dapat diaktifkan untuk mengirimkan nilai 0 atau 1. Ekspresi logika Boolean ini terbentuk ketika gerbang sirkuit digabungkan untuk membentuk sirkuit yang kompleks. Keluaran akhir Q ditentukan berdasarkan operasi yang dilakukan pada masukan A, B, dan C.

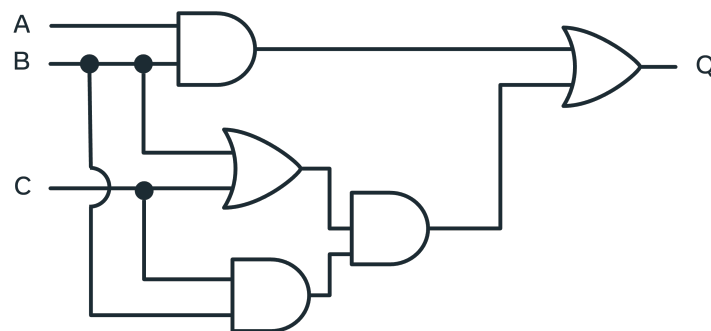


Figure 11:

1.20 CPU dan Memori

- CPU (Central Processing Unit): Unit pemrosesan pusat yang mengolah informasi dan instruksi. CPU bekerja lebih cepat daripada kecepatan transfer informasi ke dalamnya.
- Memori: Terdiri dari blok-blok memori yang menyimpan informasi dan instruksi. Kapasitas memori mengacu pada jumlah bytes yang dapat disimpan oleh komputer.

1.21 Jenis-jenis Memori

- Cache Memory: Memori termahal yang terletak dekat dengan CPU. Cache menyimpan informasi yang sering diakses untuk mempercepat proses. Jika informasi tidak ada di cache, CPU akan mencari di memori utama.
- Main Memory: Terdiri dari RAM (Random Access Memory) dan ROM (Read-Only Memory).
 - RAM: Memori yang dapat diprogram dan menyimpan data serta instruksi yang sedang digunakan. Memori ini bersifat volatile, artinya data hilang saat daya mati.
 - ROM: Memori yang hanya dapat dibaca dan tidak dapat diubah. Menyimpan instruksi penting untuk fungsi komputer.
- Secondary Memory: Memori eksternal yang dapat ditambahkan untuk meningkatkan kapasitas penyimpanan. Contohnya termasuk cloud storage, hard drive eksternal, dan memory sticks. Akses ke memori sekunder lebih lambat karena memerlukan transfer informasi ke RAM terlebih dahulu.

1.22 Pendekatan untuk Membuat Solusi di Komputer Sains

Cara mendefinisikan solusi untuk pernyataan masalah dengan mengidentifikasi masalah, merumuskan model, dan menyempurnakan solusi.

1.22.1 Mengidentifikasi masalah

Langkah pertama dalam menyelesaikan suatu masalah adalah mengartikulasikannya. **Saya perlu mencapai tujuan A, dengan menggunakan alat T, dengan batasan C.** Untuk memperjelas konsep praktik terbaik dalam merancang solusi, mari kita pertimbangkan masalah dunia nyata. Misalnya, Anda harus merancang aplikasi yang akan menghibur anak dengan permainan tebak-tebakan. Secara sekilas, ini tampak seperti tugas yang sederhana: komputer + permainan = anak yang bahagia. Namun, sebelum memulai, ada baiknya menentukan beberapa detail.

- Bagaimana anak akan berinteraksi dengan komputer?
- Pada tingkat apa pertanyaan-pertanyaan tersebut harus diajukan?
- Dari mana pertanyaan-pertanyaan tersebut dihasilkan?
- Bagaimana mereka akan disimpan?
- Bagaimana jawaban-jawaban tersebut akan diperiksa?
- Bagaimana program akan dimulai dan diakhiri?

Dalam skenario ini, masalahnya melibatkan baik perangkat keras maupun perangkat lunak. Pertama, apakah anak tersebut sudah cukup umur sehingga tidak memerlukan langkah-langkah keamanan? Dengan kata lain, penting untuk mempertimbangkan solusi ideal dengan memperhatikan bagaimana aplikasi akhir akan digunakan. Memberikan laptop mahal kepada balita mungkin bisa memberikan ketenangan sejenak, tetapi bisa saja menghabiskan gaji sebulan! Pola pikir yang sama harus diterapkan pada aplikasi yang direncanakan. Apakah pengguna akan mengakses aplikasi Anda dengan sinyal internet yang kuat? Apakah ada batasan lain, seperti output yang harus kompatibel dengan berbagai jenis ponsel? Apakah ada sistem operasi atau browser tertentu yang harus didukung? Semua pertanyaan ini perlu dipertimbangkan sebelum Anda mulai mengembangkan aplikasi.

1.22.2 Merumuskan model

Sekarang setelah ada gambaran tentang beberapa batasan proyek, solusi potensial dapat diusulkan. Bayangkan anak tersebut sudah cukup usia untuk menggunakan komputer, aplikasi hanya perlu dijalankan di laptop, semua penyimpanan dilakukan secara lokal (sehingga tidak ada masalah koneksi internet), dan input dapat dilakukan menggunakan keyboard di baris perintah. Bagus, sekarang proyek ini dapat dieksplorasi lebih lanjut.

Setelah menentukan ruang lingkup proyek, saatnya mempertimbangkan apa yang harus dilakukan oleh komputer. Algoritma dapat didefinisikan sebagai urutan instruksi yang tepat untuk memecahkan masalah. Berguna untuk terlebih dahulu menggeneralisasi masalah dan kemudian menggambarkan urutan instruksi yang diperlukan secara lebih rinci sebelum mengimplementasikan kode. Ini seperti mengambil pendekatan algoritmik untuk memecahkan masalah. Seorang programmer sering kali mendapatkan intuisi tentang bagaimana solusi seharusnya terlihat dengan terlebih dahulu menggambarkan masalah menggunakan pseudocode. Ini dapat berupa deskripsi teks yang merinci persyaratan.

```
Generate a question
Take in user input
Compare input with answer
Return a result.
```

Figure 12:

Menyusun langkah-langkah dalam daftar adalah langkah awal yang baik. Sekarang, pertimbangkan cara menghasilkan pertanyaan yang sesuai dengan usia. Menghasilkan pertanyaan secara dinamis dari ensiklopedia online atau sumber online lainnya mungkin memakan waktu. Sebagai alternatif, Anda dapat mengidentifikasi sumber pertanyaan dan jawaban yang sudah dikompilasi.

Kedua, bagaimana masukan pengguna diambil? Telah ditetapkan bahwa keyboard adalah opsi yang layak, tetapi seberapa mahir anak dalam mengetik? Membandingkan solusi dengan teks memiliki tantangannya sendiri. Apakah harus cocok secara langsung, apakah ejaan, tanda baca, dan huruf besar akan dipertimbangkan? Potensial, pertanyaan dapat berupa pilihan ganda. Meningkatkan kesadaran akan semua pertimbangan ini adalah tujuan dari merumuskan pernyataan masalah secara jelas daripada langsung melompat ke solusi.

1.22.3 Menyempurnakan solusi

Setelah mempertimbangkan dengan matang, Anda menyadari bahwa pendekatan pengetahuan dasar akan menimbulkan terlalu banyak hambatan, sehingga proyek ini didefinisikan ulang sebagai permainan menebak angka. Permainan ini cocok untuk semua usia dan tidak memerlukan sumber daya eksternal. Selain itu, permainan ini seharusnya dapat diimplementasikan dengan sebagian besar bahasa pemrograman.

```
Generate a question
Take in user input
Compare input with answer
Return a result.
```

Figure 13:

Setelah menjelaskan persyaratan umum, kita dapat menguraikan detail-detailnya. Di bawah ini adalah diagram alir pseudocode, yang dapat memberikan wawasan lebih lanjut tentang cara terbaik untuk merumuskan solusi.

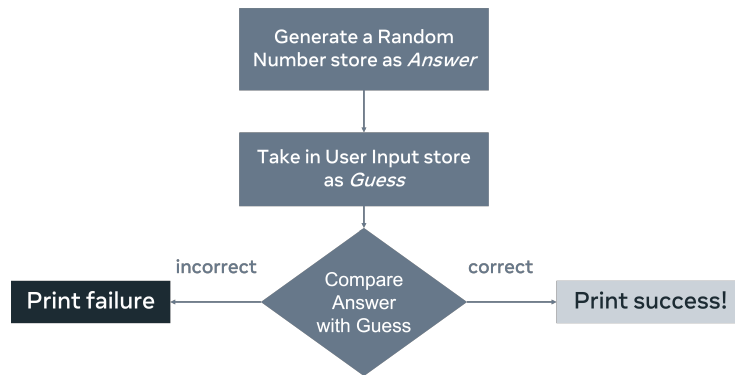


Figure 14:

Menganalisis diagram alir mengidentifikasi pertimbangan tambahan. Apakah program akan dijalankan hanya sekali? Apa yang dapat ditambahkan untuk meningkatkan pengalaman pengguna? Menampilkan pesan kegagalan mungkin bukan output yang ideal. Sebagai gantinya, opsi yang mungkin adalah memberikan kesempatan lain untuk menebak. Pertimbangan tambahan lainnya adalah menyediakan petunjuk yang dapat digunakan untuk mengarahkan pengguna ke jawaban yang benar. Dan, Anda harus memutuskan apa yang terjadi jika pengguna memasukkan nilai non-angka ke dalam program.

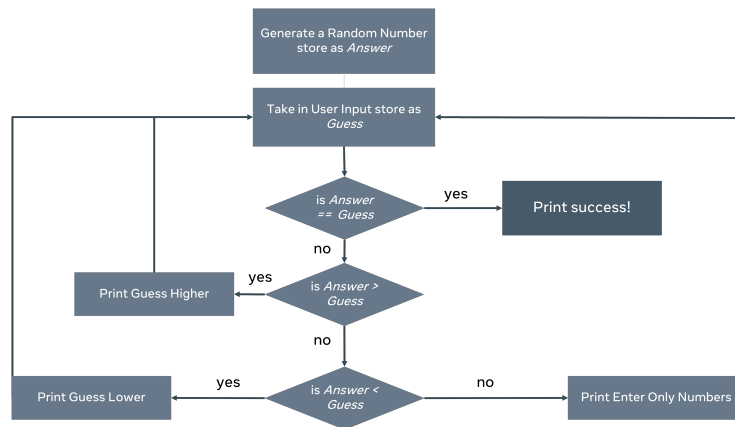


Figure 15:

Sekarang setelah berbagai kondisi telah dijelaskan, kita dapat memilih bahasa pemrograman dan mengimplementasikan solusi. Dengan mengambil pendekatan sistematis dalam pemecahan masalah, Anda telah mengidentifikasi beberapa masalah potensial sebelum proyek dimulai dan dapat melakukan penyesuaian arah. Setelah mengevaluasi solusi, cakupan proyek mungkin perlu diubah. Alih-alih membuat permainan untuk diimplementasikan oleh anak, pro-

gram tersebut mungkin akan menciptakan lingkungan sehingga anak dapat membuat permainan sendiri, menggunakan deskripsi yang jelas sebagai panduan.

1.23 Evaluasi Time Complexity

- Kompleksitas Waktu (Time Complexity): Ukuran seberapa lama algoritma akan berjalan berdasarkan ukuran input. Ini penting untuk memastikan aplikasi memberikan respons dalam waktu yang dapat diterima.
- Notasi Big-O (Big-O Notation): Metode untuk menggambarkan kompleksitas waktu algoritma. Contoh notasi yang umum termasuk $O(1)$, $O(\log n)$, $O(n)$, dan $O(n^2)$.

1.24 Contoh Kompleksitas Waktu

- $O(1)$: Algoritma yang memerlukan waktu konstan untuk menyelesaikan tugas, terlepas dari ukuran input. Contoh: mencetak item pertama dalam array.
- $O(n)$: Algoritma yang memerlukan waktu linear, di mana waktu yang dibutuhkan sebanding dengan ukuran input. Contoh: mencari nilai dalam array dengan memeriksa setiap item.
- $O(\log n)$: Algoritma yang lebih efisien daripada $O(n)$, di mana waktu yang dibutuhkan meningkat secara marginal dengan bertambahnya input. Contoh: pencarian biner (binary search).
- $O(n^2)$: Algoritma dengan kompleksitas kuadratik, di mana waktu yang dibutuhkan berlipat ganda untuk setiap elemen dalam array. Contoh: algoritma yang melibatkan dua loop yang masing-masing berjalan sebanyak n kali.

1.25 Kasus Terbaik, Terburuk, dan Tengah

Walau $O(n)$ melekat pada loop, tapi loop tidak selalu menghasilkan kompleksitas waktu tersebut, dan dapat memiliki 3 opsi:

- Kasus Terbaik (Best Case): Situasi di mana algoritma menyelesaikan tugas dengan waktu paling cepat. Contohnya ketika apa yang dicari sudah ditemukan di awal array, maka notasinya adalah $O(1)$
- Kasus Terburuk (Worst Case): Situasi di mana algoritma memerlukan waktu paling lama untuk menyelesaikan tugas. Ini jika yang dicari tidak ditemukan, maka loop akan berjalan sampai akhir, dan inilah $O(n)$
- Kasus Rata-rata (Average Case): Waktu yang diharapkan untuk menyelesaikan tugas dalam situasi umum. Ini jika loop sudah bisa diberhentikan di tengah array, $O(n/2)$.

1.26 Pertanyaan untuk Evaluasi

Sebelum memulai, tanyakan pada diri sendiri: "Berapa banyak komputasi yang digunakan solusi saya? Apakah ada cara yang lebih baik?"

1.27 Bekerja dengan Kompleksitas Waktu

1.27.1 Pendahuluan

Evaluasi kinerja aplikasi memastikan bahwa kode yang ditulis baik dan sesuai dengan tujuannya. Pertanyaannya adalah bagaimana kita mengevaluasi efisiensi? Saat mengukur listrik, kita menggunakan kilowatt-jam, yang berarti berapa banyak kilowatt yang akan digunakan oleh suatu perangkat jika dijalankan selama satu jam. Perangkat tersebut tidak selalu dijalankan selama satu jam, dan mungkin memiliki persyaratan yang berbeda tergantung pada pengaturan yang digunakan, ini lebih merupakan pedoman umum untuk mengevaluasi biaya.

Saat mengevaluasi solusi pemrograman, notasi Big-O digunakan. Jadi, notasi Big-O adalah kilowatt-jam dalam evaluasi kode. Notasi ini dapat diterapkan untuk mengukur berapa lama waktu yang dibutuhkan oleh sepotong kode atau berapa banyak ruang yang akan digunakan di memori. Tidak semua prosesor berjalan dengan kecepatan yang sama, jadi daripada mengukur waktu aplikasi, kita menghitung jumlah instruksi yang diinisialisasi oleh aplikasi.

1.27.2 Mana Pengukuran yang Benar Merefleksikan Kemungkinan Eksekusi yang Paling Cepat dari Sebuah Kode?

Mari kita telusuri pengukuran mana yang mencerminkan eksekusi kode tercepat yang mungkin.

O(1) Anda menggunakan algoritma waktu konstan yang membutuhkan waktu O(1) (O-dari-satu) untuk dihitung. Hal ini menentukan bahwa hanya diperlukan satu kali perhitungan untuk menyelesaikan suatu tugas. Contohnya adalah mencetak suatu elemen dari sebuah array dengan indeks.

```
# An array with 5 numbers
array = [0,1,2,3,4]

# retrieve the number found at index location 3
print(array[3])
```

Dalam hal ini, terlepas dari berapa banyak nilai yang ada dalam array, pendekatan ini memiliki kompleksitas Big-O sebesar satu. Artinya, menjalankan kode ini dianggap memiliki kompleksitas O(1).

O(n) Selanjutnya, mari kita lihat contoh dari O(n). Menggunakan array yang sama, ditulis pernyataan if yang mencari angka 5. Untuk memastikan bahwa 5

tidak ada di sana, pernyataan tersebut harus memeriksa setiap elemen dalam array.

```
# An array with 5 numbers
array = [0,1,2,3,4]

if 5 in array:
    print("five_is_alive")
```

Dalam contoh di atas, tidak ada angka 5, jadi tidak ada hasil cetak. Untuk memastikan hal ini, dilakukan lima pemeriksaan pada array ini. Karena input $n = 5$, kode ini dikatakan memiliki kompleksitas Big-O sebesar $O(n)$. Untuk memahami hal ini dengan lebih baik, mari kita perpanjang array menjadi 10 dan menghilangkan angka 5.

```
# an array with 10 numbers
array = [0,1,2,3,4,6,7,8,9,10]

if 5 in array:
    print("five_is_still_alive")
```

Dengan memperluas array menjadi 10 bilangan bulat, jumlah perhitungan kini menjadi 10. Hal ini masih disebut $O(n)$ karena ukuran inputnya adalah 10, yang merupakan jumlah pemeriksaan yang harus dilakukan sebelum program berakhir.

$O(\log n)$ Pencarian ini kurang intensif daripada $O(n)$ tetapi lebih banyak kerja daripada $O(1)$. $O(\log n)$ adalah pencarian logaritmik dan akan meningkat seiring penambahan input baru, tetapi penambahan input ini hanya memberikan peningkatan marginal. Contoh yang sangat baik dari hal ini adalah pencarian biner. Pencarian biner dibahas lebih detail di bagian selanjutnya dalam kursus ini.

Sekarang, bayangkan bermain permainan tebak-tebakan dengan petunjuk berikut: terlalu tinggi, terlalu rendah, atau benar. Anda diberikan rentang 100 hingga 1. Anda mungkin memutuskan untuk mendekati masalah ini secara sistematis. Pertama, Anda menebak 50 – terlalu tinggi. Jadi, Anda menebak 25 – yang juga terlalu tinggi. Anda mungkin memilih untuk melanjutkan ke 12 atau 13. Yang terjadi di sini adalah Anda membagi dua ruang pencarian dengan setiap tebakan.

Jadi, meskipun input untuk fungsi ini adalah 100 menggunakan pendekatan pencarian biner, Anda seharusnya dapat menemukan jawabannya dalam kurang dari 5 atau 6 tebakan. Solusi ini memiliki kompleksitas waktu $O(\log n)$. Bahkan jika n (rentang angka yang dimasukkan) sepuluh kali lebih besar, jumlah tebakan yang diperlukan tidak akan sepuluh kali lipat.

Berikut ini adalah rincian langkah-langkah tersebut pada array.

```
array = [0,1,2,3,4,6,7,8,9,10]
```

```

print("##Step_One")
print("Array")
print(array)
midpoint = int(len(array)/2)
print("the midpoint at step one is: " , array[midpoint])

print()

print("##Step_Two")
array = array[:midpoint] # 6 is the midpoint of the array
print("Array")
print(array)
# running this shows the numbers left to check
# is 5 < 3
# no
# so discard the left hand side

# so the array is halved again
midpoint=int(len(array)/2)
print("the midpoint is:", array[midpoint])

print()
print("##Step_Three")
array = array[midpoint:] # so the array is halved at the
midpoint
print(array)
# check for the midpoint
midpoint=int(len(array)/2)
print("the midpoint is: " , array[midpoint])
# is 4 < 5
# yes look to the right

print()
print("##Step_Four")
print(array[midpoint:])
# check for the midpoint
array = array[midpoint:] # so the array is halved at the
midpoint
midpoint=int(len(array)/2)

print()
print("##Step_Five")
array = array[midpoint:]
print(array)
print("only one value to check and it is not 5")

```

Outputnya:

```

##Step One
Array
[0, 1, 2, 3, 4, 6, 7, 8, 9, 10]
the midpoint at step one is: 6

##Step Two
Array
[0, 1, 2, 3, 4]
the midpoint is: 2

##Step Three
[2, 3, 4]
the midpoint is: 3

##Step Four
[3, 4]

##Step Five
[4]
only one value to check and it is not 5

```

Anda akan menyadari bahwa untuk menentukan apakah angka 5 ada, diperlukan 5 langkah. Itu adalah skor Big-O sebesar $O(5)$. Anda dapat melihat bahwa ini lebih besar dari $O(1)$ tetapi lebih kecil dari $O(n)$. Sekarang, apa yang terjadi jika array diperluas menjadi 100? Saat mencari angka dalam array berukuran 10, diperlukan 5 tebakan. Memeriksa array berukuran 100 tidak akan membutuhkan 50 tebakan; paling banyak hanya 10. Demikian pula, jika daftar diperluas menjadi 1000, tebakan hanya akan mencapai 15-20.

Dari ini, kita dapat melihat bahwa kompleksitasnya bukan $O(1)$ karena jawaban tidak langsung diperoleh. Kompleksitasnya juga bukan big- $O(n)$ karena jumlah tebakan tidak meningkat seiring dengan ukuran n dari array. Oleh karena itu, di sini dikatakan bahwa kompleksitasnya adalah $O(\log(n))$.

Untuk mendapatkan pemahaman yang lebih mendalam tentang bagaimana nilai logaritma hanya mengalami kenaikan secara bertahap, lihat tabel logaritma hingga 100.000.000. Perspektif ini menunjukkan bahwa $O(\log n)$ hanya menimbulkan biaya pemrosesan yang minimal. Melakukan pencarian biner pada array dengan n nilai apa pun, dalam skenario terburuk, akan selalu menghasilkan jumlah perhitungan yang tercantum dalam kolom nilai logaritma.

1	n	Log Values
2	10	3
3	100	7
4	1000	10
5	10000	13
6	100000	17
7	1000000	20
8	10000000	23
9	100000000	27

Figure 16:

$O(n^2)$ memiliki beban komputasi yang tinggi. Ini adalah kompleksitas kuadratik, artinya jumlah operasi yang diperlukan tumbuh secara proporsional dengan kuadrat ukuran input. Sebuah array berukuran n akan memiliki jumlah operasi total sekitar n^2 . Cara yang baik untuk memvisualisasikan ini adalah dengan mempertimbangkan bahwa Anda memiliki berbagai array. Mengikuti contoh sebelumnya, mari kita jelajahi kode berikut:

```
new_array=[] # an array to hold all of the results
# array with five numbers
array = [0,1,2,3,4]
for i in range(len(array)): # the array has five values, so
    this is n=5
    for j in range(len(array)): # still the same array so n
        = 5
        new_array.append(i*j) # every computation made is
            stored here

print(len(new_array)) #how big is this new array ?
```

Lingkaran pertama akan sama dengan jumlah elemen input, n . Lingkaran kedua juga akan memeriksa jumlah elemen input, n . Oleh karena itu, kompleksitas

keseluruhan dari pendekatan ini dapat dikatakan sebagai $n * n$, yang merupakan n^2 (n kuadrat). Untuk mengetahui berapa banyak perhitungan yang dilakukan, Anda harus mencetak jumlah kali n digunakan dalam lingkaran seperti di bawah ini.

```
n = 5 #size of array
print(n*n) # how big is this new array ?
```

$O(n^2)$ memiliki beban komputasi yang tinggi. Ini adalah kompleksitas kuadratik, artinya jumlah operasi yang diperlukan tumbuh secara proporsional dengan kuadrat ukuran input. Sebuah array berukuran n akan memiliki jumlah operasi total sekitar n^2 . Cara yang baik untuk memvisualisasikan ini adalah dengan mempertimbangkan bahwa Anda memiliki berbagai array. Mengikuti contoh sebelumnya, mari kita jelajahi kode berikut:

1.27.3 Representasi Visual dari Sebuah Masalah

Di bawah ini adalah representasi grafis yang menunjukkan hubungan antara n dengan jumlah perhitungan yang diperlukan.

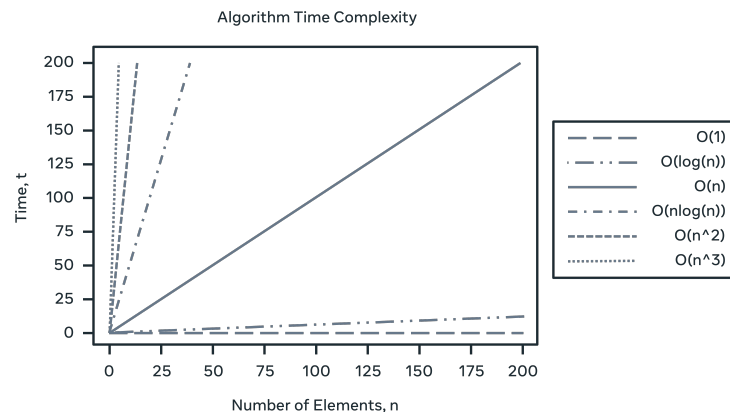


Figure 17:

Seperti yang dapat Anda lihat, waktu terbaik yang harus dituju adalah $O(1)$; $O(\log n)$ masih sangat baik. $O(n)$ masih bisa diterima, sedangkan $O(n^2)$ tidak terlalu baik.

1.27.4 Kasus buruk, Kasus baik, dan Kasus rata-rata

Tentu saja, tidak selalu mungkin untuk mengetahui berapa lama suatu pendekatan akan memakan waktu. Ketika melihat contoh loop awal, terdapat pencarian yang dilakukan untuk elemen yang tidak ada. Anda dapat mengatakan bahwa mencari dalam loop memakan waktu $O(n)$, tetapi hal ini tidak selalu berlaku.

Pertimbangkan bahwa elemen yang dicari adalah elemen pertama dalam array. Maka waktu kembalinya akan cukup baik dalam $O(1)$ waktu! Dalam contoh yang diberikan, setiap elemen harus dicari sebelum menentukan bahwa elemen tersebut tidak ada: $O(n)$ waktu. Kasus tengah adalah ketika elemen tersebut ditemukan di tengah loop: $O(n/2)$. Saat mengevaluasi suatu pendekatan, tiga definisi digunakan: kasus terbaik, kasus terburuk, dan kasus rata-rata.

1.28 Konsep Dasar

- Kompleksitas Ruang: Mengukur seberapa banyak memori yang dibutuhkan oleh algoritma untuk menyelesaikan masalah, sering kali menjadi trade-off dengan waktu eksekusi.
- Big-O Notation: Notasi yang digunakan untuk menggambarkan kompleksitas ruang, seperti $O(1)$, $O(n)$, $O(\log n)$, dan lainnya, di mana n merujuk pada ukuran input.

1.29 Jenis Ruang dalam Kompleksitas

- Auxiliary Space: Ruang tambahan yang diperlukan untuk menyimpan data sementara saat algoritma berjalan. Ini mencakup variabel sementara dan struktur data tambahan.
- Input Space: Ruang yang diperlukan untuk menyimpan data yang dimasukkan ke dalam fungsi atau algoritma.

1.30 Contoh Perhitungan

- Dalam Java, sebuah integer memerlukan 4 bytes, dan array kosong memerlukan 12 bytes untuk header dan 4 bytes untuk padding. Jadi, untuk array integer dengan ukuran 4, total memori yang dibutuhkan adalah 32 bytes.
- Jika ukuran array menjadi 8 integer, total memori yang dibutuhkan menjadi 48 bytes, tetapi ukuran auxiliary space tetap sama jika tidak terpengaruh oleh ukuran input.

1.31 Pertimbangan Desain

- Penting untuk mempertimbangkan penggunaan memori saat merancang aplikasi. Menghindari penyalinan array atau struktur data yang kompleks dapat meningkatkan efisiensi ruang.
- Menggunakan struktur data yang lebih sederhana ketika memungkinkan dapat mengurangi overhead memori.

2 Pengenalan ke Struktur Data

2.1 Pengantar Struktur Data

- Struktur Data: Model objek yang memungkinkan penyimpanan dan pengorganisasian data dalam memori komputer.
- Mutable vs Immutable:
Mutable adalah struktur yang dapat diubah setelah dibuat. Immutable adalah struktur yang tidak dapat diubah setelah dibuat.

2.2 Jenis Struktur Data

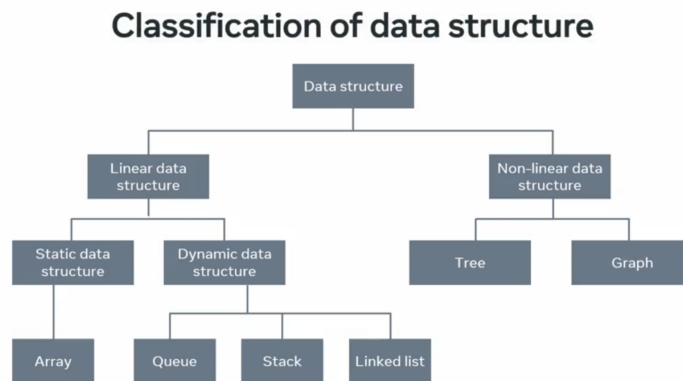


Figure 18:

Linear Structures: Elemen disimpan secara berurutan. Contoh:

- Array: Kumpulan elemen dengan tipe data yang sama, diakses menggunakan indeks (misalnya, `array[4]` untuk elemen kelima).
- List: Mirip dengan array, tetapi dapat menyimpan tipe data yang berbeda.

Nonlinear Structures: Elemen tidak disimpan secara berurutan. Contoh:

- Trees: Struktur yang memungkinkan penyimpanan data dalam hierarki.
- Graphs: Struktur yang menghubungkan elemen dengan cara yang tidak teratur.

2.3 Pertimbangan dalam Memilih Struktur Data

- Kinerja: Memilih struktur data yang tepat dapat mempengaruhi kemajuan proyek.

- Penggunaan Memori: Mempertimbangkan alokasi dan de-alokasi memori untuk menghindari kebocoran memori. Memory Leak adalah situasi di mana memori tidak digunakan dan tidak didealokasi, yang dapat menyebabkan aplikasi kehabisan memori.

2.4 Strings

String adalah fitur yang terdapat dalam setiap bahasa pemrograman. String adalah urutan terurut dari karakter atau simbol yang diapit oleh tanda kutip tunggal atau ganda yang sesuai. Sebagian besar bahasa pemrograman mendukung karakter ASCII dasar dan representasi Unicode, meskipun terdapat banyak cara tambahan untuk mewakili string. Setiap karakter ASCII biasanya menempati satu byte memori, sementara karakter Unicode dapat menempati lebih banyak byte, tergantung pada enkoding yang digunakan (UTF-8, UTF-16, dll).

2.4.1 Representasi String

Ada perbedaan penting dalam cara setiap bahasa pemrograman mewakili dan mendukung string. Semua bahasa pemrograman mendukung operasi dasar seperti membuat, menyalin, dan mengaitkan string ke variabel. Namun, apakah string dapat dimodifikasi secara langsung bergantung pada apakah bahasa tersebut menganggap string sebagai mutable atau immutable. Selain itu, tindakan sehari-hari meliputi:

- Menggabungkan (menghubungkan string teks).
- Menambahkan string.
- Mencari substring.
- Mengelola kumpulan string.

Saat memproses string, banyak bahasa pemrograman memungkinkan Anda untuk melakukan operasi aljabar pada string tersebut. Misalnya, `String_A == String_B`, yang akan mengembalikan nilai True atau False atau `String_A < String_B`, yang akan menentukan mana yang lebih dulu secara alfabetis. Beberapa bahasa pemrograman, seperti PHP, memungkinkan variabel untuk langsung disisipkan ke dalam string. Bahasa pemrograman lain mungkin mengharuskan variabel ditempatkan dalam kurung kurawal, seperti JavaScript.

Escape Flags memungkinkan penyisipan simbol dalam sebuah string. Misalnya, bayangkan Anda ingin menyisipkan kutipan dalam sebuah string:

```
String = "the man said \"two more pints please\" to the barman"
```

Di sini, penggunaan tanda backslash \ memastikan bahwa string tetap mempertahankan tanda kutip dalam kalimat. Simbol escape lainnya dapat berupa #, %, ', atau tanda kutip ganda di dalam string yang ditandai dengan tanda

kutip tunggal. Meskipun implementasinya mungkin berbeda, pendekatan umum dalam menangani string tetap sama.

2.4.2 Kegunaan Umum

Salah satu bidang yang banyak berurusan dengan string adalah Pemrosesan Bahasa Alami (NLP). Cara string diencode saat dibaca dari berbagai sumber seperti Twitter, file PDF, dokumen teks, atau Reddit dapat menimbulkan masalah yang tidak terduga. Masalah ini dapat menyebabkan masalah debugging yang kompleks jika aplikasi memproses representasi simbol yang tidak biasa yang memengaruhi hasil.

Saat menulis string ke dalam program, tokenisasi sering diterapkan. Tokenisasi berarti mengubah string menjadi array string yang lebih kecil. Di sini, Anda akan mengidentifikasi pemisah dan memecah string menjadi segmen yang dipisahkan oleh pemisah tersebut. Misalnya, Anda mungkin memecah paragraf menjadi kalimat menggunakan titik (.), atau memecah kalimat menjadi kata-kata individu menggunakan spasi (" "). Namun, Anda harus berhati-hati saat melakukan tokenisasi teks. Pertimbangkan teks di bawah ini yang memiliki tiga simbol titik (.).

```
At 3.30 I went to the shop and spent $40.40.
```

Secara konvensional, Anda akan membacanya sebagai satu kalimat, tetapi titik-titik yang digunakan untuk menunjukkan waktu dan jumlah uang berfungsi sebagai pemisah, yang membagi kalimat menjadi tiga bagian.

Tokenisasi pada data yang telah diproses dan diformat tidak terlalu menantang. Cara umum untuk menyimpan representasi string yang diformat adalah melalui Comma Separated Values (CSV) atau Tab Separated Values (TSV). Pemrosesan string semacam itu dikenal sebagai parsing, dan array yang dikembalikan akan sama dengan jumlah pemisah yang ditemukan, dengan pemisah tersebut dihapus.

Berikut adalah contoh file CSV yang berisi kolom usia, nama, dan warna rambut. Perhatikan bahwa tidak ada koma di akhir. Kecuali dinyatakan lain, simbol baris baru juga berfungsi sebagai pemisah.

```
age, name, hair color
24, John, black
25, Tony, brown
34, Brian, blue
29, Mary, red
43, Martin, yellow
```

Figure 19:

2.4.3 Imutabilitas

Konsep penting yang perlu dipertimbangkan saat bekerja dengan string adalah apakah string tersebut bersifat mutable atau immutable. Mutability mengacu

pada kemampuan Anda untuk mengubah string setelah string tersebut dibuat. Beberapa bahasa pemrograman, seperti Ruby dan PHP, memungkinkan string diubah setelah dibuat. Namun, lebih umum menggunakan immutability, seperti dalam Java, C#, JavaScript, Python, dan Go.

Ada beberapa alasan mengapa bahasa pemrograman membuat string bersifat immutable. Pertama, hal ini dapat mengurangi penggunaan memori; alih-alih membuat variabel yang berisi string, sebuah string pool dirancang untuk mewakili semua string yang digunakan. Pendekatan immutable mengurangi penggunaan alokasi memori dengan membuat semua instance **"string"** mengacu ke satu lokasi. Jadi, ketika terjadi perubahan pada string, misalnya menjadi **"strings"**, alih-alih mengubah nilai **"string"**, variabel tersebut diarahkan ke instance lain dari **"strings"**. Atau diarahkan ke contoh immutable lain yang mewakili **"strings"**, yang kemudian ditambahkan ke string pool.

Immutability adalah alat penghemat memori yang sangat efektif. Bayangkan membaca kata-kata yang sudah ada daripada membuat lokasi memori untuk setiap kata, termasuk yang berulang. Menggunakan himpunan unik mengurangi ruang yang dibutuhkan. Kelemahannya adalah jika aplikasi Anda terus-menerus mengubah teks, setiap perubahan akan menimbulkan penalti memori.

2.5 Integers

Bilangan bulat digunakan untuk menyimpan nilai numerik. Bilangan bulat dapat berupa bilangan bertanda (menyimpan bilangan positif dan negatif) atau bilangan tak bertanda (hanya menyimpan bilangan positif). Bilangan bulat direpresentasikan dalam sistem biner, dan representasi dasarnya umumnya memerlukan 4 byte. Tergantung pada sistem dan bahasa pemrograman, bilangan bulat juga dapat menempati 1, 2, atau 8 byte.

2.5.1 Representasi Integer

Ada beberapa cara untuk mewakili bilangan bulat dalam sistem biner. Salah satu contoh yang umum adalah sistem tanda-besaran. Jika bilangan bulat diwakili dalam sistem biner, bagaimana cara membedakan antara nilai positif dan negatif? Sistem tanda-besaran mengusulkan penggunaan indikator di bagian paling kiri angka biner untuk menunjukkan polaritas.

Integer	Sign-magnitude representation
2	0010
1	0001
+0	0000
-0	1000
-1	1001
-2	1010

Figure 20:

Integer tidak dapat mewakili pecahan. Untuk itu, digunakan bilangan decimal atau float. Untuk mewakili integer, digunakan jumlah byte tetap, ukurannya dapat ditentukan dalam beberapa bahasa pemrograman. Ada pendekatan standar untuk mewakili bilangan floating-point, seperti decimal dan float, yang disebut standar IEEE 754, yang menetapkan serangkaian standar umum untuk mewakili bilangan-bilangan ini.

Beberapa bahasa pemrograman tingkat tinggi seperti Python dan JavaScript mengikuti pendekatan ini dan mengenkapsulasi inisialisasi bilangan bulat ke dalam representasi tetap ini. Hal ini memudahkan kerja dengan angka dalam bahasa pemrograman dinamis tingkat tinggi ini, tetapi menghilangkan kemampuan untuk menyesuaikan pendekatan Anda guna mengoptimalkan penggunaan memori.

2.5.2 Alokasi Memori

Bahasa pemrograman bertipe statis lainnya seperti C++, Rust, Ada, dan C memungkinkan Anda untuk menyesuaikan ukuran dalam memori. C++ memungkinkan Anda untuk menyesuaikan tipe integer Anda. Menggunakan unsigned short int hanya akan memakan 2 byte. Penghematan ini dapat bertambah jika aplikasi Anda mencakup banyak angka positif yang tidak memerlukan presisi hingga tingkat pecahan.

Rust melangkah lebih jauh dan memungkinkan instansiasi integer tak bertana 1 byte (dengan tipe data `u8`, tipe data yang serupa juga ada di C++ dengan nama `unsigned char`). Bilangan bulat terbatas ini dapat menyimpan bilangan bulat dari 0 hingga 255. Meskipun rentang ini mungkin tidak terlihat besar, jika Anda bekerja dengan piksel, Anda akan bekerja dengan tuple nilai dalam rentang ini. Pemrosesan gambar adalah proses yang sangat memakan sumber daya CPU; kemampuan menyimpan 8 piksel dengan biaya 1 piksel standar adalah penghematan yang besar.

2.5.3 Class Pembungkus

Selain bilangan bulat primitif yang dilambangkan dengan `int`, Java memungkinkan Anda untuk membungkus nilai bilangan bulat primitif ke dalam kelas

pembungkus Integer. Hal ini memungkinkan berbagai metode untuk menangani bilangan bulat, seperti konversi dari string ke double, perbandingan, ukuran maksimum dan minimum, dan sebagainya. Kelas Integer bersifat immutable, yang membuatnya aman untuk digunakan dalam thread. Fungsi tambahan dan keamanan ini menimbulkan biaya memori, dan objek Integer akan membutuhkan 16 byte memori untuk disimpan.

2.5.4 Kesimpulan

Ada kebebasan yang sangat besar dalam mengontrol cara penyimpanan data Anda di memori. Meskipun bahasa pemrograman tingkat tinggi yang dinamis dapat berjalan dengan cepat, hal ini mungkin tidak memberikan hasil yang optimal jika aplikasi Anda menjadi sangat memakan sumber daya. Misalkan Anda bekerja dengan Arduino atau perangkat elektronik hobi lainnya. Dalam hal ini, ruang menjadi sangat berharga, dan kebebasan untuk menyesuaikan mungkin menjadi perbedaan antara aplikasi yang berfungsi dan yang tidak berfungsi. Namun, jika Anda mengembangkan aplikasi yang memerlukan jumlah memori yang besar, menggunakan bilangan bulat dengan cepat dan tidak perlu khawatir tentang detailnya mungkin menjadi pilihan yang tepat.

2.6 Booleans

2.6.1 Pernyataan Kondisional

Ekspresi Boolean dapat menggunakan berbagai operator relasional.

>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
==	Equal to
!=	Not equal to

Figure 21:

Ini memungkinkan Anda untuk memeriksa beberapa data sebelum menjalankan opsi kode yang berbeda. Oleh karena itu, ekspresi Boolean sering disebut sebagai kondisional. Gabungkan ini dengan pernyataan kondisional seperti if, dan Anda memiliki implementasi awal kecerdasan buatan (AI).

Pernyataan kondisional meliputi: if, else, else if, while, dan sebagainya. Blok kondisional memungkinkan Anda untuk menjalankan satu set instruksi dalam satu kondisi dan set instruksi lain jika kondisinya berbeda. Pertimbangkan diagram berikut.

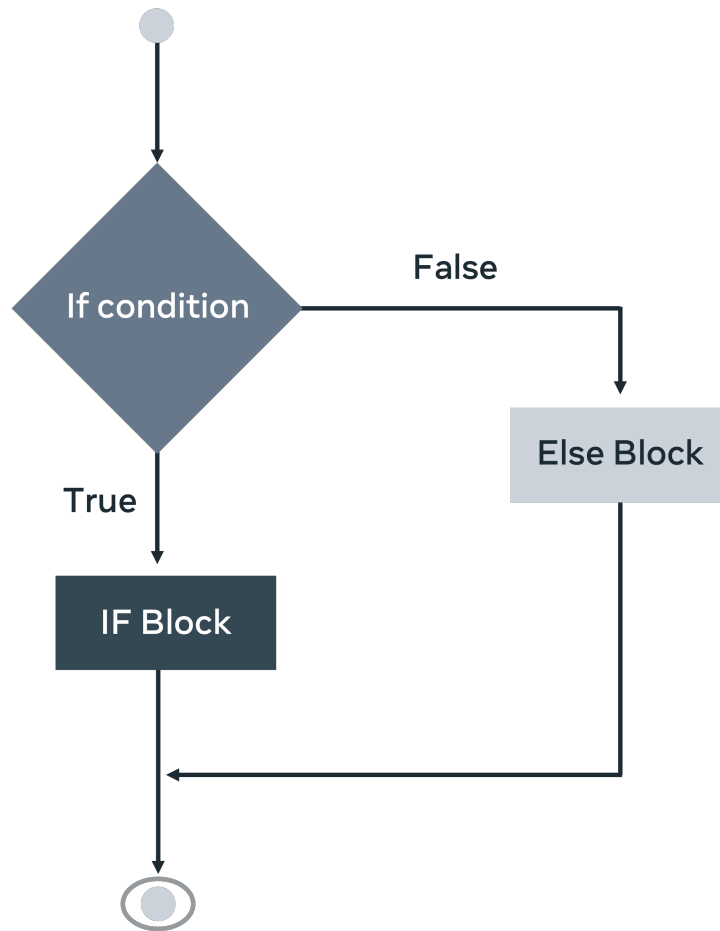


Figure 22:

Diagram yang sama diwakili dalam kode Python berikut:

```
if right > wrong:
    doTheRightThing()
elif wrong > right:
    doTheWrongThing()
else:
    keepResearching()
```

Dengan menggunakan ekspresi Boolean sebagai indikator, Anda dapat menggunakan operator relasional untuk menentukan baris kode mana yang akan dieksekusi. Dalam contoh ini, komputer mengevaluasi antara benar dan salah. Hal ini memiliki berbagai aplikasi dan sering digunakan. Akhirnya, jika tidak ada kondisi yang terpenuhi, Anda dapat menggunakan blok kode penangkap (catch-all). Roombas (Roomba yang dimaksud adalah *robot vacuum cleaner* buatan

iRobot. Mereka sering dipakai sebagai contoh nyata bagaimana konsep boolean logic dan relational operators) adalah contoh utama bagaimana kode ini dapat digunakan dalam kehidupan sehari-hari. Alih-alih benar dan salah, Anda akan memiliki sensor dasar yang menentukan motor mana yang akan diaktifkan berdasarkan jarak dan dampak.

2.6.2 Operator Logika

Selain itu, Anda mungkin ingin memperluas cakupan aplikasi Anda dengan menggunakan operator logika.

Logical operators	
	Logical OR
&&	Logical AND
!	Logical NOT

Figure 23:

Ekspresi logika ini dapat digabungkan dengan ekspresi Boolean untuk memberikan kode Anda variasi yang lebih besar.

```
if condition_1 || condition_2:
    doActionOne()
elif condition_1 && condition_2:
    doActionTwo()
elif !condition_1:
    doActionFour()
else:
    waitForInstruction()
```

Dalam kode ini, empat kondisi diperiksa. Pertama, ambil `condition_1` atau `condition_2`, dan mari kita gunakan contoh Roomba. Jika deteksi jarak atau deteksi benturan benar, maka tindakan pertama mungkin adalah berhenti dan mundur. Jika ada deteksi jarak dan deteksi benturan, maka akan memicu alarm. Jika tidak ada deteksi jarak, maka akan terus mengirim daya ke motor depan. Akhirnya, jika tidak ada kondisi yang terpenuhi, harus ada kode fail-safe yang siap diaktifkan.

Penggunaan logika Boolean merupakan tulang punggung desain sirkuit. Ini adalah cara menginterpretasikan operator Boolean secara bersamaan. Bentuk-bentuk dalam diagram di bawah ini adalah gerbang yang diaktifkan ketika operator Boolean tertentu dipicu. Sinyal dikirim melalui kabel (digambarkan oleh garis masuk dan keluar). Operator AND menyatakan bahwa jika kedua kabel benar, maka lanjutkan jalur eksekusi ini. Pada saat yang sama, operator NAND menentukan bahwa hanya dua pernyataan salah yang akan memicu reaksi.

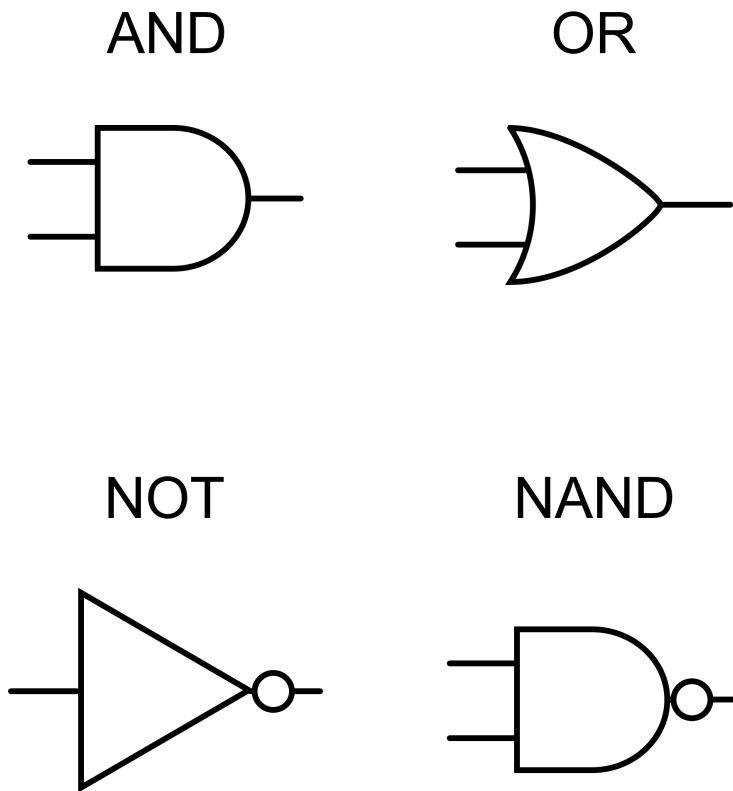


Figure 24:

Secara bersama-sama, mereka dapat digunakan untuk mengambil keputusan yang kompleks dan membuat perangkat beroperasi dengan cara yang beragam, tergantung pada masukan dari sensor-sensor tersebut.

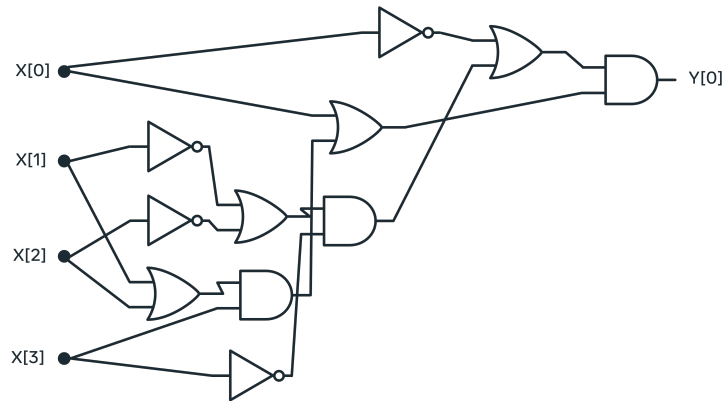


Figure 25:

Boolean expression menghasilkan nilai true/false dari perbandingan.

Logical expression digunakan untuk menggabungkan atau memanipulasi boolean expression.

Untuk menyimpulkan, ekspresi Boolean dapat bernilai benar atau salah, dan ekuivalennya dalam sistem biner adalah 0 atau 1. Sama seperti dalam sistem biner, Anda mungkin berpikir bahwa hanya memiliki dua nilai akan membatasi kemampuannya. Namun, ekspresi Boolean dapat mencapai tingkat kompleksitas yang cukup tinggi ketika digabungkan dengan konstruksi matematis lain seperti pernyataan kondisional dan operator logika. Ekspresi Boolean dapat digunakan dalam program komputer Anda untuk menentukan operasi mana yang harus dijalankan secara otomatis dan menjadi dasar dari diagram sirkuit.

2.7 Arrays

Semua bahasa pemrograman memiliki bentuk array, tetapi cara kerjanya mungkin sedikit berbeda. Misalnya, beberapa bahasa menentukan bahwa array harus berisi jenis elemen yang sama, seperti array string atau array integer.

Di sisi lain, bahasa pemrograman lain memungkinkan Anda menyimpan elemen campuran dalam sebuah array. Fungsi array juga dapat berbeda. Terkadang, array dapat menjadi objek yang lengkap dengan fungsi yang kompleks. Di saat yang sama, Anda dapat menganggapnya sebagai tipe penyimpanan untuk tipe primitif; Anda kemudian dapat menerapkan operasi dan fungsi secara eksternal.

2.7.1 Menginisialisasi Arrays

Anda dapat membuat array secara statis atau dinamis. Dalam bahasa pemrograman statis, array disimpan di tumpukan (stack) dan jenis array harus ditentukan. Bahasa pemrograman dinamis menawarkan fleksibilitas lebih, terkadang

mengharuskan Anda menentukan ukuran array tanpa perlu menentukan jenisnya sebelum digunakan. Instansi semacam itu disimpan di tumpukan (heap). Array memiliki indeks, yaitu nilai berurutan yang dimulai dari 0 dan bertambah hingga akhir array. Saat suatu item diperlukan, nama array diberikan, diikuti oleh kurung siku yang berisi lokasi indeks.

```
array_name[0]
```

Contoh di atas akan mengambil item pertama dalam array. Anda dapat menggunakan angka apa pun untuk mengembalikan elemen di lokasi indeks tersebut. Menentukan angka yang lebih besar dari ukuran array akan menimbulkan kesalahan out-of-bounds. Tindakan umum yang akan Anda lakukan dengan array adalah mengiterasi melalui array dan memeriksa elemen-elemennya.

```
n <- size of array arr
FOR (i <- 0; i < (n-1); i <- (i+1)) DO
    process element arr[i]
END
```

Bentuk umum dari perulangan `for` sama di semua pendekatan: Anda memerlukan ukuran array, bilangan bulat `i` yang bertambah pada setiap iterasi, dan perulangan `for` umum untuk sintaksis. Tampilannya mungkin sedikit berbeda, tetapi mekanisme dasarnya sama. Mulai dari 0, lalu setiap elemen dalam array dan lakukan sesuatu. Sebuah array selalu memiliki cara untuk mengakses ukurannya. Beberapa contohnya adalah `.size()`, `len(array)`, dan `.length()`.

2.7.2 Mengelola Array di dalam Memori

Perbedaan mendasar mengenai array dalam berbagai bahasa pemrograman terletak pada tempat penyimpanannya. Anda dapat menyimpan suatu item di memori heap atau memori stack. Memori stack dibuat saat menjalankan suatu fungsi. Memori ini dibuat khusus untuk fungsi tersebut dan dihapus setelah eksekusi selesai. Item yang disimpan di alokasi memori ini hanya tersedia untuk fungsi tersebut. Di sisi lain, memori heap dibuat selama eksekusi instruksi dan tersedia untuk semua.

Anda harus berhati-hati saat mengubah elemen dalam lingkungan statis. Saat sebuah array diinstansiasi, ruang memori sebesar ukuran inisialisasi yang ditentukan dibuat di tumpukan panggilan. Tumpukan panggilan dibuat untuk menjalankan tujuan saat sebuah fungsi dipanggil. Mengubah array dan mengembalikannya dari tumpukan dapat menyebabkan memori rusak, karena tumpukan dibuang setelah fungsi selesai.

Bahasa pemrograman dinamis menghindari masalah ini dengan menyimpan array di heap. Dengan demikian, array tetap utuh meskipun fungsi berakhir dan stack dibuang. Stack, secara alami, menyimpan blok memori yang berurutan, sehingga memudahkan akses informasi. Heap kurang terorganisir dan dapat memakan waktu lebih lama untuk mengakses elemen-elemennya. Sekali lagi,

Anda perlu mempertimbangkan trade-off antara ukuran dan kenyamanan, atau aksesibilitas versus kecepatan.

Secara umum, ketika program selesai menggunakan array, memori tersebut dibebaskan dan menjadi tersedia untuk penggunaan lain. Cara pengelolaan pembebasan memori dan pengumpulan sampah (garbage collection) bergantung pada bahasa pemrograman yang dipilih dan hanya dapat meningkatkan kinerja jika dilakukan dengan efektif. Pengelolaan memori yang buruk dapat menyebabkan kebocoran memori, yang mungkin menyebabkan aplikasi Anda crash saat dipanggil berulang kali.

Dua konsep terkait memori yang mungkin Anda temui adalah salinan dangkal (shallow copy) dan salinan dalam (deep copy). Salinan dangkal tidak membuat salinan array tetapi mengembalikan lokasi indeks. Salinan dalam akan membuat instance baru dari array. Membuat salinan dangkal mengoptimalkan penggunaan memori; namun, Anda harus memastikan bahwa Anda tidak secara tidak sengaja melakukan perubahan yang tidak terduga pada array yang dibagikan oleh dua variabel.

Selain itu, penting untuk memahami bahwa heap tidak secara otomatis bersifat global. Memori di heap hanya dapat diakses selama masih ada pointer atau referensi yang menunjuk ke sana. Jika pointer hilang, memori tersebut tetap terpakai tetapi tidak bisa lagi digunakan, yang dikenal sebagai memory leak. Inilah sebabnya bahasa seperti C dan C++ mewajibkan programmer untuk secara manual membebaskan memori dengan `free()` atau `delete`. Sebaliknya, bahasa modern seperti Java, Python, dan JavaScript menggunakan garbage collector yang secara otomatis melacak referensi dan membebaskan memori yang sudah tidak digunakan.

Walaupun begitu, kebocoran logis tetap bisa terjadi jika programmer menyimpan referensi terlalu lama, misalnya menaruh array besar di variabel global yang tidak pernah dihapus. Untuk menghindari hal ini, sebaiknya variabel yang hanya diperlukan sementara ditempatkan di dalam fungsi, sehingga lingkungannya terbatas dan memori dapat dibersihkan setelah fungsi selesai. Dengan cara ini, manajemen memori menjadi lebih aman dan efisien.

2.7.3 Menggabungkan Dua Array

Matriks adalah array dua dimensi (atau array yang terdiri dari array) yang dapat berfungsi seperti tabel. Matriks dapat digunakan untuk mewakili baris dan kolom. Setiap elemen dalam matriks memiliki koordinat 2D (x,y). Di sini, x merujuk pada baris, dan y merujuk pada kolom. Seperti sebelumnya, Anda akan menggunakan kurung siku untuk mengakses elemen dalam array dua dimensi. Namun, seperti yang ditunjukkan di bawah ini, Anda memerlukan dua lokasi indeks untuk sebuah matriks.

<code>Matrix[x][y]</code>

Sebuah matriks akan memiliki bentuk persegi dengan jumlah kolom dan baris yang sama. Anda tidak perlu menggunakan bentuk ini, tetapi jika Anda

memilih untuk tidak mempertahankan keseragaman, pastikan loop Anda bertindak sesuai.

```
int[][] matrix = new int[5][7]
for (int i = 0; i < matrix.length; i++) {
    for(int j = 0; j < matrix[i].length; j++)
    {
        doSomethingNeo(matrix[i][j])
    }
}
```

Dalam contoh di atas, sebuah array matriks 2D diinisialisasi untuk menyimpan bilangan bulat. Koleksi luar dapat berisi lima elemen (array bilangan bulat). Semua array dalam memiliki kapasitas 7. Array pertama dalam array luar dipilih dengan nilai *i*, ukurannya ditentukan (7), dan setiap elemen dari array dalam tersebut kemudian diteruskan ke metode. Perhatikan bagaimana ukuran array dalam diakses secara dinamis menggunakan atribut `.length`; hal ini memastikan tidak terjadi kesalahan out-of-bounds selama iterasi.

2.8 Objects

Objek merupakan blok bangunan dasar dari semua kode dan memainkan peran sentral dalam pemrograman berorientasi objek (OOP).

2.8.1 Definisi

Objek adalah konsep pemrograman yang berarti suatu struktur memiliki baik keadaan (state) maupun perilaku (behavior). Di sini, perilaku berkaitan dengan kemampuan objek untuk melakukan suatu tindakan. Secara konkret, hal ini berkaitan dengan memanggil metode dari suatu objek. Contohnya adalah memanggil metode `sort` dari suatu array. Hal ini mengakibatkan pengaturan ulang elemen-elemen array sehingga mereka terorganisir satu sama lain.

Status atau atribut suatu objek berkaitan dengan informasi tentang objek tersebut. Jika Anda membuat kelas orang dan menginstansiasikannya menjadi objek, contoh status (atau atribut) mungkin usia atau nama orang tersebut. Perilaku kemudian mungkin berkaitan dengan tindakan yang diperlukan dari objek orang ini, seperti berlari atau menendang.

Class sering dijelaskan sebagai cetak biru untuk sebuah Object. Dengan demikian, sebuah object dapat dijelaskan sebagai instansi dari sebuah class. Penggunaan objek yang paling umum adalah dalam Pemrograman Berorientasi Objek (OOP), di mana kode dikapsulkan ke dalam objek, dan objek-objek ini kemudian berinteraksi satu sama lain.

2.8.2 Contoh

Salah satu keunggulan utama menggunakan objek untuk menginstansiasi class adalah Anda hanya perlu membuat satu templat untuk cara objek tersebut berinteraksi. Setelah itu, Anda bebas membuat beberapa instansi dari kelas ini

yang dapat berinteraksi satu sama lain. Bayangkan sebuah tim sepak bola dengan 11 pemain. Hanya satu kelas yang perlu didefinisikan yang dapat menampung karakteristik kecepatan, kelincahan, dan tingkat kerja yang bervariasi. Hal ini dapat secara signifikan mengurangi jumlah kode yang diperlukan dalam pembuatan permainan sepak bola.

Karakteristik kelas seperti kecepatan dan kelincahan dikenal sebagai variabel instance dan dapat dikatakan berkaitan dengan keadaan objek. Tidak ada unit di luar objek yang akan berbagi dengan instance ini. Dua pemain dapat memiliki instance kebugaran. Mengubah instance ini pada satu pemain tidak akan memengaruhi instance kebugaran yang terdapat pada objek lain. Anda dapat mengubah instance kelas melalui konstruktor atau metode internal objek. Dalam Java, instansiasi kelas yang umum dilakukan adalah sebagai berikut:

```
Player player1 = new Player(agility = 54, speed = 88,
    fitness = 90);

Player player2 = new Player(agility = 90, speed = 64,
    fitness = 83);
```

Dalam contoh ini, class pemain membuat dua objek, player1 dan player2. Variabel di dalam kelas kemudian diatur secara berbeda tergantung pada nilai yang terdapat di dalam kurung kurawal. Diperlukan metode untuk mengubah variabel tersebut. Selama pertandingan, seorang pemain mungkin menjadi lelah. Metode dapat dipanggil untuk mengurangi nilai variabel agar mencerminkan kondisi tersebut.

```
player1.set_fitness(80)
```

Ini akan mengubah tingkat kebugaran player1 tetapi tidak akan memengaruhi pemain lain. Metode yang digunakan untuk mengubah instance atribut umumnya disebut getter dan setter.

Kontrol lebih lanjut terhadap objek dapat dilakukan melalui metode lain yang terdapat pada objek tersebut. Perintah seperti

```
player1.kick()
```

akan menyebabkan objek yang ditugaskan ke player1 melakukan prosedur menendang bola.

Kelas pemain memungkinkan pembuatan tim dengan kemampuan yang bervariasi yang dapat bermain. Konsep lain yang terkait dengan penggunaan objek adalah pewarisan. Bayangkan ada kebutuhan untuk membuat objek manusia lain di lapangan, seperti wasit, yang tidak bermain tetapi tetap melakukan tindakan terkait permainan, seperti berlari dan penampilan umum. Anda mungkin ingin menggunakan kembali sebagian kode dari kelas pemain tanpa membuat objek pemain baru. Di sini, Anda akan membuat kelas yang menampung semua atribut umum yang ingin dipertahankan, dan setiap objek dapat mewarisi atribut tersebut, seperti yang ditunjukkan.

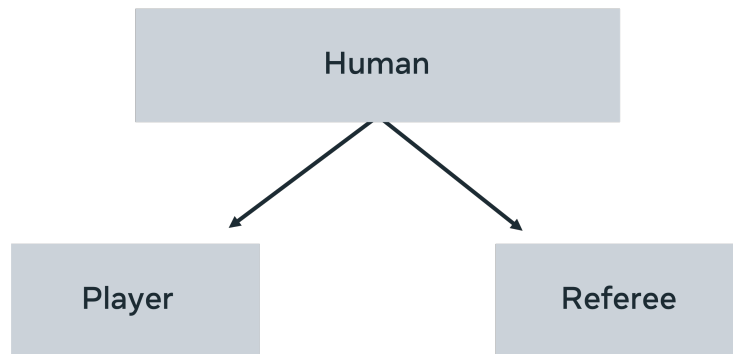


Figure 26:

Oleh karena itu, metode standar seperti berlari, merasa lelah, dan mengikuti bola dapat ditemukan pada manusia, tetapi cara kerja metode-metode ini dalam kaitannya dengan kelas Pemain dan kelas Wasit dapat dibedakan. Sebuah konsep yang memiliki satu konsep umum (manusia) yang dapat muncul dalam bentuk yang berbeda (pemain, manajer, wasit) dikenal sebagai polimorfisme. Satu bentuk, banyak wujud. Contoh klasiknya adalah penggunaan bentuk. Bentuk utama akan mencakup konsep area, diameter, dan tinggi. Kemudian, setiap instance bentuk (persegi, segitiga, lingkaran) akan menerapkan implementasi aktual bagaimana atribut-atribut ini terlihat dalam keadaan masing-masing.

2.9 Lists

Lists adalah objek yang menyimpan data dan memiliki metode bawaan. Mereka dapat berisi elemen dengan tipe yang sama atau campuran.

Implementasi: Lists dapat diimplementasikan menggunakan arrays atau linked lists.

- Array-based List: Koleksi terurut yang dibangun menggunakan arrays. Memiliki batasan ukuran awal dan dapat memperluas ukuran saat diperlukan, tetapi ini dapat mahal dalam hal waktu komputasi.
- Linked List: Terdiri dari node yang menyimpan data dan pointer ke node berikutnya. Memungkinkan pertumbuhan dinamis dengan menambahkan node baru, yang membuatnya cepat untuk menyimpan data besar.

Sintaks List:

```
list = [1, 2, 3] (Python)
```

2.10 Sets

Sets adalah koleksi elemen yang tidak terurut dan hanya menyimpan elemen unik. Jika elemen yang sama ditambahkan, tidak ada perubahan pada data.

Pencarian Cepat: Sets menggunakan hash tables untuk menyimpan elemen, yang memungkinkan pencarian dilakukan dalam waktu $O(1)$.

- Hashing Function: Algoritma yang memetakan data ke nilai tetap yang unik. Ini memungkinkan pencarian cepat, tetapi dapat mengalami clash-ing jika dua nilai berbeda menghasilkan nilai hash yang sama.

Sintaks Set:

```
set = {1, 2, 3} (Python)
```

2.11 List dan Set dalam Bahasa Pemrograman yang Berbeda

Set dan List adalah tipe data bawaan dalam banyak bahasa pemrograman dengan tema umum yang berlaku untuk sebagian besar di antaranya. Dalam implementasi sebenarnya, terdapat beberapa perbedaan halus.

2.11.1 Mutabilitas

Konsep ketidakubahaan telah dibahas sebelumnya. Secara konkret, hal ini berkaitan dengan cara struktur data dibuat. Objek yang tidak dapat diubah tidak dapat diubah setelah dibuat, sementara objek yang dapat diubah dapat diubah setelah dibuat. Kumpulan (set) adalah contoh struktur yang tidak dapat diubah dalam kebanyakan bahasa pemrograman. Namun, objek yang ditambahkan ke set dapat diperlakukan secara berbeda tergantung pada bahasa pemrograman. Dalam JavaScript, Anda dapat menambahkan objek yang dapat diubah ke dalam set, hal ini tidak diperbolehkan dalam Python. Memahami perbedaan mendasar ini akan memberikan wawasan tentang cara bahasa pemrograman menggunakan kumpulan.

2.11.2 Kenapa Set Cepat?

Alasan di balik kecepatan yang dimiliki oleh set dalam menemukan suatu item terletak pada arsitektur dasarnya. Set menyimpan nilai menggunakan pendekatan **hashing**. Meng-hash sesuatu berarti mengambilnya dan menghasilkan output unik darinya. Hal ini dilakukan dengan menerapkan algoritma yang mengubah input menjadi output alfanumerik sederhana (kata dan huruf). Setiap kali algoritma melihat input, ia akan selalu menghasilkan output alfanumerik yang sama. Output ini kemudian digunakan dalam tabel hash, yang menggunakan hash unik untuk menentukan di mana item disimpan dalam memori. Oleh karena itu, dengan satu perhitungan, kita dapat mengetahui apakah nilai tersebut ada dalam memori atau tidak. Alih-alih mencari setiap elemen dalam daftar, cukup terapkan fungsi hashing dan periksa apakah item tersebut digunakan.

Oleh karena itu, set sangat cepat, tetapi alasan kecepatannya juga menjadi alasan mengapa nilai yang disimpan tidak dapat diubah. Jika Anda membuat

hash untuk suatu elemen dan menyimpannya sesuai dengan hash tersebut, lalu mengubah item tersebut, hash tidak akan lagi dapat menemukannya. Inilah salah satu alasan mengapa Python hanya mengizinkan penyimpanan objek immutable yang tidak dapat diubah dalam set. Tidak semua bahasa pemrograman menawarkan perlindungan ini. Misalnya, JavaScript memang mengizinkan penyimpanan objek yang dapat diubah. Oleh karena itu, perlindungan objek diserahkan kepada pengguna saat menggunakan set dalam JavaScript. Jika Anda ingin mengubah nilai yang dapat diubah dalam JavaScript, disarankan untuk mengekstrak dan menghapus objek yang dapat diubah tersebut sebelum memasukkannya kembali ke dalam set sebagai elemen baru.

Seperti Python, Kotlin tidak mengizinkan elemen untuk diubah setelah ditambahkan ke dalam sebuah set. Hal ini berarti akses ke set tersebut bersifat read-only. Jika Anda ingin set yang dapat diubah, Kotlin memiliki jenis set lain yang disebut `MutableSet`. Set dalam Kotlin sangat fleksibel dan menyediakan berbagai metode bawaan seperti `max`, `min`, `sum`, dan `average`. Hal ini membuatnya sangat berguna saat mencari item tertentu dalam kelompok angka.

2.11.3 List

Jenis koleksi penting lainnya yang perlu diketahui adalah daftar (list). Daftar menyimpan elemen dalam urutan tertentu yang dapat diakses menggunakan indeks. Indeks hanyalah angka yang diberikan ke daftar untuk menunjukkan bahwa elemen yang berada di posisi angka tersebut harus dikembalikan. Indeks adalah cara elemen dicari dalam daftar. Jika seseorang ingin mengetahui apakah sesuatu ada dalam daftar, mereka harus melakukan pencarian dan memeriksa setiap item, sehingga lebih lambat daripada himpunan (set). Selain itu, daftar memungkinkan Anda menyimpan berbagai jenis elemen di dalamnya. Tidak ada perbedaan antara apakah suatu item bersifat mutable atau immutable. Daftar juga memungkinkan Anda menyimpan duplikat item. Ada berbagai jenis daftar yang diimplementasikan secara berbeda.

Namun, poin fundamental yang perlu diingat adalah bahwa daftar memungkinkan Anda menyimpan item yang berulang dan item dengan jenis yang berbeda. Daftar bersifat terurut, artinya urutan item akan tetap terjaga saat dimasukkan. Hal ini berbeda dengan himpunan, yang mungkin menyimpan item di lokasi yang berbeda dari tempat awalnya dimasukkan.

Memahami perbedaan halus ini dapat membantu saat memutuskan jenis struktur data mana yang ingin Anda pilih. Baik himpunan (set) maupun daftar (list) berguna, tetapi karena keduanya berperilaku berbeda, mereka akan lebih berguna dalam situasi tertentu daripada yang lain. Memastikan bahwa daftar atau himpunan adalah pilihan yang paling sesuai dalam suatu situasi adalah kuncinya.

2.12 Stack

Stack adalah struktur data linear yang mengikuti prinsip First-In, Last-Out (FILO) atau Last-In, First-Out (LIFO). Artinya, elemen terakhir yang ditam-

bahkan adalah yang pertama diambil.

Metode Utama:

- Push: Menambahkan elemen ke atas stack.
- Pop: Menghapus elemen dari atas stack.
- isEmpty: Memeriksa apakah stack kosong.
- isFull: Memeriksa apakah stack sudah penuh.
- Peek: Melihat elemen teratas tanpa menghapusnya.

2.12.1 Contoh Penggunaan Stack

Misalnya, saat menggunakan Ctrl+Z untuk membatalkan tindakan terakhir dalam aplikasi, tindakan tersebut diambil dari atas stack.

2.13 Queue

Queue adalah struktur data yang mengikuti prinsip First-In, First-Out (FIFO). Artinya, elemen pertama yang ditambahkan adalah yang pertama diambil.

Metode Utama:

- Enqueue: Menambahkan elemen ke belakang queue.
- Dequeue: Menghapus elemen dari depan queue.
- isEmpty: Memeriksa apakah queue kosong.
- isFull: Memeriksa apakah queue sudah penuh.

2.13.1 Contoh Penggunaan Queue

Contoh nyata adalah antrean di restoran cepat saji, di mana pelanggan pertama yang datang adalah yang pertama dilayani.

2.14 Stack dan Queue dalam Bahasa Pemrograman yang Berbeda

Tumpukan (stack) adalah struktur data dasar yang menentukan cara data disimpan dalam suatu aplikasi. Elemen dalam tumpukan harus mengikuti kebijakan masuk atau keluar last-in-first-out (LIFO). Berbagai bahasa pemrograman memiliki implementasi yang berbeda, namun secara umum, hasilnya tetap sama. Antrian (queue) sangat mirip dengan tumpukan; namun, urutan masuknya berbeda, mengutamakan kebijakan first-in-first-out (FIFO).

2.14.1 Stacks

Tumpukan (stack) adalah contoh dari tipe data abstrak. Dalam ilmu komputer, ini berarti bahwa beberapa karakteristik esensial harus diterapkan saat mengimplementasikannya, tetapi tidak ada versi bawaan yang dapat Anda impor secara langsung. Untuk tumpukan, prinsip pentingnya adalah LIFO (last-in-first-out).

Analogi untuk tumpukan adalah membayangkan tumpukan piring di atas papan cuci. Setiap kali piring dibersihkan, piring tersebut diletakkan di atas piring sebelumnya. Untuk mengeringkannya, setiap piring harus diambil dari bagian atas. Jadi, piring terakhir yang ditambahkan ke tumpukan dikeringkan terlebih dahulu, dan piring pertama di tumpukan dikeringkan terakhir.

Penggunaan umum tumpukan (stack) adalah untuk menyimpan riwayat penjelajahan browser Anda. Setiap kali Anda menekan tombol kembali, halaman yang sebelumnya dikunjungi dimuat dan, ke depannya, dibaca ke dalam tumpukan.

Implementasi umum tumpukan dalam JavaScript dilakukan dengan menggunakan array, memastikan bahwa array tersebut berperilaku seperti tumpukan. Ketika Anda menggunakan struktur data untuk membangun struktur data lain, Anda dikatakan menggunakan *Container Adapter*. Di sini, array adalah kontainer-nya, dan adaptasi memaksa array tersebut berperilaku seperti stack. Fitur penting dari tumpukan adalah push, pop, peek, dan count. Push menambahkan item ke bagian atas tumpukan. Menggunakan array berarti bahwa count harus selalu mengetahui di mana item terakhir ditempatkan di array. Demikian pula, memiliki count untuk array penting untuk metode pop, karena pop hanya mengembalikan item terakhir yang dimasukkan ke tumpukan.

2.14.2 Queues

Queue, seperti stack, adalah struktur data linier yang mempertahankan urutan di mana item dimasukkan. Struktur linier menyimpan item sesuai urutan penambahan. Sama seperti tumpukan, antrian memiliki implementasi ketat tentang cara item ditambahkan dan dihapus. Sementara tumpukan menggunakan pendekatan LIFO (Last In, First Out), antrian menggunakan pendekatan first-in-first-out (FIFO). Oleh karena itu, item pertama yang ditambahkan ke antrian akan menjadi item pertama yang dihapus. Seperti pelanggan yang mengantri untuk berbelanja, antrian menerapkan kebijakan yang menekankan waktu kedatangan. Hal ini memiliki manfaat nyata dalam implementasi seperti penjadwalan CPU dan disk, di mana menangani tugas sesuai urutan kedatangan sangat penting.

Antrian adalah tipe data abstrak dalam Swift, artinya untuk menggunakan fungsionalitas antrian, Anda harus terlebih dahulu membuat struktur yang berfungsi seperti itu. Dua istilah penting terkait antrian adalah enqueue (menambahkan item ke antrian) dan dequeue (menghapus item dari bagian belakang daftar). Pendekatan termudah untuk membuat antrian adalah menggunakan array sebagai adaptor kontainer. Karena antrian hanya mengambil elemen dari bagian depan antrian, waktu pencarian selalu $O(1)$, sehingga ap-

likasi yang dibangun dengan pendekatan ini dapat mengharapkan layanan yang sangat cepat. Seperti halnya tumpukan, metode yang tersedia untuk antrian dibatasi, dengan *peak*, *pop*, *push*, dan *size* sebagai yang paling penting.

2.14.3 Kesimpulan

Memilih struktur data yang tepat untuk tugas yang tepat merupakan keputusan pemrograman yang sangat penting, karena hal ini dapat memengaruhi keseluruhan proyek. Anda akan memilih tumpukan (*stack*) ketika ada kebutuhan untuk melacak urutan masuknya suatu item, dan untuk mengembalikannya dengan cara yang sangat spesifik. Antrian (*queue*) digunakan untuk menjaga urutan, tetapi akan mengikuti aturan yang berbeda dari tumpukan. Akhirnya, jika Anda ingin struktur data di mana Anda dapat menetapkan prioritas, maka antrian prioritas (*priority queue*) layak dipertimbangkan.

2.15 Struktur Umum Trees

Tree adalah struktur data yang terdiri dari nodes yang saling terhubung. Setiap node dapat menjadi **parent** atau **child**. Node tanpa anak disebut **leaf node**.

Root adalah Node teratas dalam tree. Node yang terhubung ke root disebut child node. Node yang memiliki parent yang sama disebut sibling.

2.16 Terminologi Penting pada Trees

- **Path:** Rangkaian node yang terhubung. Menggambarkan cara tercepat untuk berpindah dari satu node ke node lainnya.
- **Depth:** Jumlah edges dari parent ke root, atau jalur terpanjang.
- **Height:** Jumlah edges dari node teratas ke node terdalam dalam struktur.
- **Size:** Total jumlah nodes dalam tree.

2.17 Jenis-jenis Trees

- **Binary Trees:** Setiap node memiliki maksimal dua child. Node kiri memiliki nilai lebih kecil, sedangkan node kanan memiliki nilai lebih besar.
- **B Trees dan B-plus Trees:** Variasi dari tree yang digunakan untuk penyimpanan data yang efisien.
- **AVL Trees:** Tree yang seimbang, menjaga tinggi tree agar tetap optimal.

2.18 Keuntungan Menggunakan Trees

- Menyimpan data secara hierarkis, memungkinkan pengambilan informasi yang lebih cepat.

- Efisien dalam menambah dan menghapus data karena cara implementasinya yang fleksibel.
- Dapat digunakan untuk memodelkan sistem file, hierarki kelas dalam pemrograman, atau struktur organisasi.

2.19 Metode Traversal

- Depth-First: Mengunjungi setiap node dari atas ke bawah secara berurutan.
- Breadth-First: Mencari setiap node pada level yang sama sebelum naik ke level berikutnya.

2.20 Trees dalam Bahasa Pemrograman yang Berbeda

Pohon adalah tipe data abstrak (ADT) yang terdapat dalam banyak bahasa pemrograman. Seperti yang dibahas dalam bacaan lain, ADT adalah cetak biru tentang bagaimana struktur data akan terbentuk. Hal ini berkaitan dengan batasan dan persyaratan yang diterapkan pada struktur data untuk memastikan bahwa struktur tersebut selalu beroperasi dengan cara yang sama. Hal ini sangat berguna bagi programmer yang berpindah antara beberapa bahasa pemrograman, karena memberikan ekspektasi yang jelas tentang bagaimana struktur data akan beroperasi.

Prinsip dasar utama dari pohon adalah bahwa ia mengandung sejumlah node, node akar, dan sejumlah node daun. Node daun adalah node yang tidak terhubung di dasar pohon. Node akar selalu berada di bagian atas; setiap nilai berasal dari node ini. Pohon selalu disusun dalam bentuk hierarki tertentu.

2.20.1 Implementasi Tree

Ada banyak jenis pohon; mungkin yang paling sederhana dan umum adalah **Binary Tree**. Pohon biner memiliki sifat-sifat berikut:

- Setiap node memiliki maksimal dua node anak.
- Setiap node harus memiliki kunci agar dapat diidentifikasi dengan mudah.
- Nilai yang lebih kecil dari node ditempatkan di node anak kiri, dan nilai yang lebih besar ditempatkan di node anak kanan.

Untuk membuat struktur data Tree, Anda harus memastikan adanya root (node awal) dan metode untuk menentukan node-node berikutnya. Setiap node harus mengandung referensi ke node kiri dan kanan agar pohon dapat dijelajahi. Hal ini dapat dicapai dengan membuat **class** yang memiliki tiga atribut (key, lokasi node kiri, dan lokasi node kanan). Class ini memerlukan tiga metode tambahan agar dapat berfungsi sebagai pohon. Metode-metode tersebut meliputi:

- Metode pencarian: Penting agar pohon dapat diinterogasi untuk mengetahui keberadaan atau ketidakadaan informasi.
- Metode penyisipan: Seperti yang telah disebutkan sebelumnya, menyisipkan node pada pohon melibatkan penentuan posisi yang tepat dan menempatkannya di sebelah kiri nilai tertinggi terdekat.
- Metode penghapusan: Metode ini akan menghapus suatu item dari pohon. Operasi ini menimbulkan tantangan tambahan saat diterapkan pada pohon karena sifat terhubungnya pohon. Pertimbangkan bahwa pohon terdiri dari serangkaian node yang terhubung. Oleh karena itu, menghapus satu node secara sembarangan dapat merusak koneksi dalam pohon. Oleh karena itu, saat mengimplementasikan metode ini, selain menghapus node, perlu memeriksa semua node anak dan memastikan koneksi baru dibuat dengan node nilai tertinggi berikutnya.

2.20.2 Searching pada Tree

Pendekatan yang dijelaskan di atas untuk mengkodekan pohon berlaku untuk semua bahasa pemrograman yang dibahas dalam kursus ini. Tidak ada implementasi konkret dari pohon, artinya untuk menggunakannya, Anda harus mengimplementasikan kode tersebut sendiri.

Fitur lain yang perlu diperhatikan adalah cara melakukan pencarian pada pohon. Anda dapat mengkodekan solusi yang menggunakan pencarian kedalaman-pertama (depth-first search) atau pencarian lebar-pertama (breadth-first search). Untuk memahami konsep ini dengan lebih baik, pertimbangkan bagaimana pohon tersebut diorganisasikan. Setiap level memiliki node induk yang terhubung ke dua node anak. Node anak ini, pada gilirannya, memiliki dua node anak mereka sendiri. Pencarian breadth-first akan memeriksa setiap node pada level yang sama sebelum beralih ke level yang lebih dalam. Ini dapat dibayangkan sebagai pemindaian semua node secara horizontal sebelum memeriksa level berikutnya. Pencarian depth-first akan memeriksa setiap node pada cabang hingga mencapai node akhir sebelum memeriksa cabang yang berdekatan. Ini dapat dibayangkan sebagai pemindaian vertikal pada pohon.

2.20.3 Kesimpulan

Karena pohon adalah tipe data abstrak, tidak ada metode bawaan untuk bahasa pemrograman yang digunakan. Untuk mengimplementasikan pohon, sangat penting untuk mengingat karakteristik pentingnya agar Anda dapat mengimplementasikannya dengan benar.

2.21 Apa itu Hash Table?

Hash Table adalah Struktur data yang menyimpan pasangan kunci-nilai dalam slot atau bucket. Memerlukan **Hashing Function** untuk menentukan bucket yang tepat untuk menyimpan data.

Hashing Function adalah Algoritma yang diterapkan pada kunci untuk menghasilkan angka unik yang berfungsi sebagai indeks.

2.22 Cara Kerja Hash Table

- Setiap item data yang disimpan harus memiliki key (kunci) dan value (nilai). Kunci diproses dengan hashing function untuk menghasilkan nilai tetap yang lebih kecil.
- Contoh fungsi hashing sederhana: $\text{key} \bmod 20$, yang menghasilkan nilai unik untuk kunci dari 0 hingga 9.

2.23 Koalisi dalam Hash Table

Kolisi terjadi ketika dua kunci menghasilkan nilai hash yang sama. Misalnya, $1 \bmod 20$ dan $21 \bmod 20$ keduanya menghasilkan 1.

Solusi untuk kolisi:

- Meningkatkan ukuran tabel: Setiap kali kolisi terjadi, tabel diperbesar dan nilai didistribusikan ke alamat baru.
- Linked List: Menyimpan nilai tambahan dalam linked list di titik kolisi.

2.24 Keuntungan Menggunakan Hash Table

- Kecepatan: Hash table dapat mengambil item dalam waktu $O(1)$, yang lebih cepat dibandingkan dengan pencarian di array yang memerlukan waktu $O(n)$.
- Penggunaan: Umumnya digunakan dalam cache, kamus, indeks database, dan set.

2.25 Hash Table pada Bahasa Pemrograman yang Berbeda

Kelas koleksi adalah kelas khusus untuk penyimpanan dan pengambilan data. Tergantung pada kebutuhan aplikasi Anda, pemilihan struktur data yang tepat dapat secara signifikan memengaruhi pendekatan pemrograman Anda dan efisiensi aplikasi. Sebuah hashtable menyimpan pasangan kunci-nilai dan menawarkan pencarian yang cepat. Beberapa bahasa pemrograman menyediakan implementasi bawaan, sementara yang lain memerlukan jenis koleksi lain untuk berfungsi seperti hashtable.

2.25.1 Apa itu HashTable?

Hashtable menawarkan pencarian yang sangat cepat untuk suatu aplikasi. Hal ini dicapai dengan membuat fungsi hashing yang akan menghasilkan output alfanumerik (huruf dan angka) dari input yang diberikan. Hash ini kemudian

digunakan untuk menentukan di mana dalam memori suatu data akan disimpan. Ini berarti bahwa ketika Anda ingin mengetahui apakah suatu elemen ada dalam struktur data, Anda tidak perlu memeriksa setiap item dan melakukan perbandingan. Cukup terapkan fungsi hashing dan lihat apakah item tersebut telah di-hash ke memori. Ketika mempertimbangkan bahwa sumber data mungkin memiliki jutaan entri, tidak perlu memeriksa setiap entri satu per satu merupakan penghematan waktu yang besar. Selain itu, penting untuk memastikan bahwa tabel hash tidak mengizinkan nilai null ditambahkan sebagai kunci atau nilai. Hal ini dapat menyebabkan masalah saat membandingkan nilai di dalam tabel.

Pada beberapa bahasa pemrograman, untuk mengimplementasikan instance tabel hash, diperlukan modifikasi struktur data yang sudah ada agar dapat melakukan operasi tabel hash.

2.25.2 Implementasi Hashtable pada Kotlin

Meskipun Kotlin tidak memiliki implementasi bawaan untuk hashtable, struktur data yang sangat mirip bernama hashmap didukung. Hashmaps sangat mirip dengan hashtables karena keduanya menyimpan pasangan kunci-nilai dan menggunakan hash untuk menentukan di mana di memori kunci tersebut disimpan. Ada beberapa perbedaan yang perlu diperhatikan: hashmap memungkinkan penggunaan null untuk kunci dan nilai, dan tidak aman untuk thread.

2.25.3 Apa itu thread-safe, dan bagaimana aplikasinya ke hashtable?

Threads adalah proses yang dapat dijalankan oleh komputer. Biasanya, saat Anda menyalakan komputer, sejumlah proses akan dimulai. Proses-proses ini meliputi hal-hal seperti membuka dokumen Word, Excel, aplikasi Java, dan sebagainya. Proses-proses ini sering dijalankan secara bersamaan. Untuk melakukannya, compiler akan membuat banyak thread yang dapat menjalankan kode. Jadi, thread adalah potongan kode yang dapat dieksekusi dan dapat menjalankan suatu proses. Lalu, bagaimana Anda dapat mengatakan bahwa kode tersebut thread-safe? thread-safe berarti jika Anda menulis program yang mengakses struktur data, Anda dapat menduplikasi aplikasi tersebut dan mengakses struktur data yang sama melalui thread yang berbeda tanpa menimbulkan kesalahan. Memiliki lima thread yang berbeda bekerja pada struktur data yang sama mungkin membuat kode Anda berjalan lima kali lebih cepat, tetapi hal ini hanya berguna jika informasi yang diubah dilakukan dengan benar.

2.25.4 Bagaimana thread-safe berkaitan dengan implementasi Kotlin untuk Hashtable

Salah satu fitur dari hashtables adalah bahwa mereka dapat disinkronkan. Artinya, jika lima proses yang berbeda menggunakan dan mengubah informasi yang sama dalam sebuah tabel, informasi tersebut selalu akurat. Kotlin tidak menyediakan implementasi hashtables; namun, terdapat struktur data yang

sangat mirip dan dapat diadaptasi untuk tujuan ini yang disebut hashmaps. Hashmaps memiliki implementasi kunci, nilai, dan pencarian hash yang sama, tetapi tidak aman untuk thread. Oleh karena itu, dalam mengimplementasikan hashtable di Kotlin, kita dapat menggunakan hashmap dan menambahkan kode yang memastikan sinkronisasi sehingga beberapa thread dapat mengaksesnya secara bersamaan.

2.25.5 Implementasi Hashtable pada Python

Sama seperti Kotlin, Python tidak memiliki implementasi bawaan untuk hashtable. Pada bagian sebelumnya, telah dijelaskan bahwa sebuah tabel dapat ditiru menggunakan struktur yang sudah ada bernama hashmap. Hal ini juga dapat dilakukan di Python, meskipun struktur dasar yang digunakan adalah dictionary. Dictionary adalah struktur data yang tepat untuk digunakan, karena bekerja berdasarkan prinsip yang sama dengan hashtable. Artinya, ia menyimpan pasangan kunci-nilai. Kunci-kunci tersebut dapat di-hash dan digunakan sebagai penunjuk lokasi penyimpanan nilai di memori. Hal ini berarti dictionary memiliki metode pencarian dan penyisipan yang sangat cepat. Selain itu, dictionary sudah thread-safe, sehingga tidak perlu diubah jika Anda memerlukan operasi yang dijalankan secara bersamaan.

2.26 Apa itu Heap?

Heap adalah struktur data yang dimodelkan seperti pohon tetapi berfungsi mirip dengan queue, dengan perbedaan utama dalam penugasan prioritas pada elemen.

Min Heap dan Max Heap: Heap yang memprioritaskan nilai kunci terendah disebut min heap, sedangkan yang memprioritaskan nilai kunci tertinggi disebut max heap.

2.27 Operasi Dasar Heap

- **Insert:** Menambahkan elemen ke dalam heap.
- **find_min:** Mengambil nilai minimum dari min heap.
- **delete_min:** Menghapus nilai minimum dari min heap.
- **find_max:** Mengambil nilai maksimum dari max heap.
- **delete_max:** Menghapus nilai maksimum dari max heap.

2.28 Implementasi dan Penggunaan Heap

- **Binary Tree:** Heap sering dibangun menggunakan binary tree, di mana nilai minimum disimpan di node akar.

- $O(1)$: Mengambil nilai minimum dari heap adalah $O(1)$ karena selalu berada di akar.
- $O(\log n)$: Waktu yang dibutuhkan untuk menyisipkan elemen ke dalam min heap adalah $O(\log n)$.

2.29 Aplikasi Heap

- Jadwal: Heap dapat digunakan untuk menjadwalkan tugas berdasarkan prioritas, seperti penjadwalan wawancara atau pengelolaan paket dalam jaringan.

2.30 Ringkasan Istilah terkait Heap

- Heap: Struktur data yang mengorganisir elemen berdasarkan prioritas.
- Min Heap: Heap yang memprioritaskan nilai kunci terendah.
- Max Heap: Heap yang memprioritaskan nilai kunci tertinggi.
- Binary Tree: Struktur data berbentuk pohon di mana setiap node memiliki dua anak.
- Insert: Menambahkan elemen ke dalam struktur data.
- `find_min`: Mengambil elemen dengan nilai minimum.
- `delete_min`: Menghapus elemen dengan nilai minimum.
- $O(1)$: Notasi yang menunjukkan waktu konstan untuk operasi.
- $O(\log n)$: Notasi yang menunjukkan waktu logaritmik untuk operasi.

2.31 Struktur Data Graph

Graph adalah struktur data yang terdiri dari nodes (simpul) dan edges (sisi) yang menunjukkan hubungan antar nodes.

Ada dua jenis Graph

- Weighted Graph: Graf yang memiliki nilai (weight) di antara nodes, menunjukkan biaya atau jarak antara dua nodes.
- Undirected Graph: Graf tanpa arah, di mana hubungan antara nodes tidak memiliki urutan tertentu, mirip dengan jalan dua arah.

2.32 Terminologi Graph

- Neighbors: Dua nodes yang berbatasan satu sama lain.
- Adjacent Nodes: Nodes yang terhubung melalui neighbor.
- Path: Rangkaian dua atau lebih nodes yang terhubung oleh edges.

2.33 Metode Traversal pada Graph

- Breadth-First Search (BFS): Metode yang mengunjungi semua nodes pada level yang sama sebelum melanjutkan ke level berikutnya. Menggunakan queue untuk menyimpan nodes yang akan dikunjungi.
- Depth-First Search (DFS): Metode yang mengeksplorasi setiap cabang hingga akhir sebelum kembali. Menggunakan stack untuk menyimpan nodes yang akan dikunjungi.

2.34 Aplikasi Graph

- Shortest Path: Menentukan jalur tercepat antara dua nodes, sering digunakan dalam routing paket di Internet atau aplikasi seperti Google Maps.
- Traveling Salesman Problem: Masalah untuk menemukan rute terbaik yang mengunjungi semua nodes dalam waktu terpendek, berguna dalam pengiriman paket.

2.35 Heap dan Graph dalam Bahasa Pemrograman yang Berbeda

Graf adalah struktur data non-linier yang dapat menyimpan informasi dengan cara yang memungkinkan Anda mengekstrak hubungan menarik yang terdapat dalam data. Heap dapat didefinisikan sebagai graf khusus yang memberikan prioritas khusus pada elemen teratas.

2.35.1 Terminologi

Graphs terdiri dari simpul (node) dan tepi (edge). Simpul adalah tempat penyimpanan data, sedangkan tepi adalah koneksi antara dua simpul. Berbeda dengan pohon, simpul tidak harus terhubung dan dapat exist secara independen dari simpul lain. Sebuah tepi menghubungkan dua simpul. Sisi dapat dikatakan memiliki bobot. Ini adalah nilai yang disimpan dalam koneksi yang mengindikasikan informasi tentang kekuatan koneksi antara dua simpul. Graph dapat dikatakan berarah, artinya sisi-sisinya berarah (seperti jalan satu arah) atau tidak berarah (seperti jalan dua arah), dan koneksi mengindikasikan informasi bolak-balik.

2.35.2 NetworkX

NetworkX adalah perpustakaan eksternal berbasis Python yang dapat diimpor ke lingkungan Python dan digunakan untuk memodelkan data. Ini adalah perpustakaan pihak ketiga yang ringan dan dapat beroperasi di lingkungan Python apa pun. Banyak jenis graph yang didukung, termasuk berarah, tak berarah, siklik, dan tak siklik. NetworkX juga mendukung berbagai algoritma berbasis graph, termasuk:

- Metrik jarak (jarak antara dua simpul)
- Centralitas (seberapa sentral suatu simpul dibandingkan dengan simpul lain)
- Dan deteksi kluster (berkaitan dengan subset dalam grafik)

Algoritma deteksi kluster (clique detection) akan menganalisis grafik untuk menentukan subset yang paling mungkin memiliki hubungan internal yang kuat dan hubungan eksternal yang lemah. Subset ini disebut clique karena kemiripannya dengan social clique. Ini dapat menjadi cara yang berguna untuk mengidentifikasi pelanggan yang berpotensi memiliki hubungan, berdasarkan sampel data pelanggan dan kebiasaan mereka. NetworkX dapat dimodelkan menggunakan matplotlib Python untuk memberikan wawasan tentang penemuan data. Ini digunakan sebagai aplikasi back-end oleh seorang programmer yang menganalisis data untuk mendapatkan wawasan.

2.35.3 Heaps

Heap diimplementasikan secara berbeda-beda dalam berbagai bahasa pemrograman, namun pada dasarnya merupakan graph dengan batasan tertentu. Heap mengurutkan informasi sehingga dapat dengan cepat mengembalikan nilai minimum dan maksimum. Oleh karena itu, heap menggunakan pendekatan biner, dan setiap implementasi harus memiliki maksimal dua node. Tergantung pada implementasinya, heap akan memiliki nilai terbesar atau terkecil sebagai akar. Akhirnya, setiap cabang heap akan mengikuti pola berurutan.

Heap memiliki waktu pencarian $O(1)$ karena hanya mengembalikan satu item. Nilai tertinggi atau terendah tergantung pada apakah itu heap minimum atau maksimum. Ini berarti bahwa setelah nilai ini diambil, item berikutnya ditambahkan ke node akar heap. Hal ini juga memengaruhi penambahan ke heap. Saat elemen baru ditambahkan, dimulai dari akar, elemen tersebut dibandingkan dengan setiap node hingga posisi yang benar ditentukan. Elemen-elemen di sekitarnya kemudian dipindahkan untuk memastikan elemen tersebut ditempatkan pada posisi yang tepat.

Cara alternatif yang digunakan oleh beberapa bahasa seperti Kotlin atau JavaScript, yang tidak memiliki implementasi bawaan, adalah mewakili tumpukan menggunakan daftar array. Sifat-sifat yang sama yang digunakan pada grafik diterapkan, meskipun implementasi aktualnya mungkin sedikit berbeda. Efisiensi tumpukan bergantung pada struktur data dasar yang digunakan, jadi disarankan untuk menyelidiki seberapa efisien implementasi bahasa target Anda sebelum menggunakannya.

2.35.4 Kesimpulan

Tumpukan dapat dianggap sebagai grafik khusus, dan grafik terdiri dari simpul dan tepi. Terdapat beberapa metode khusus yang tersedia untuk grafik, yang tujuannya adalah memungkinkan kita untuk menyimpulkan hubungan tertentu dari data yang disimpan di dalamnya.

3 Pengenalan ke Algoritma

3.1 Algoritma Sorting

- Pengurutan: Proses mengatur elemen dalam urutan tertentu, seperti secara alfabetis atau numerik.
- Metode Pengurutan: Berbagai cara untuk mengurutkan data, termasuk Selection Sort, Insertion Sort, dan Quicksort.

3.2 Selection Sort

- Definisi: Algoritma yang mencari elemen terkecil dalam daftar dan menukarnya dengan elemen pertama, kemudian mengulangi proses ini untuk elemen berikutnya.
- Sintaks:

```
for i in range(len(arr)):
    min_idx = i
    for j in range(i+1, len(arr)):
        if arr[j] < arr[min_idx]:
            min_idx = j
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

- Kelemahan: Memiliki kompleksitas waktu $O(n^2)$, yang membuatnya kurang efisien untuk daftar besar.

3.3 Insertion Sort

- Definisi: Algoritma yang membangun daftar terurut satu per satu dengan membandingkan elemen baru dengan elemen yang sudah terurut.
- Sintaks:

```
for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1
    while j >= 0 and key < arr[j]:
        arr[j + 1] = arr[j]
        j -= 1
    arr[j + 1] = key
```

- Kelemahan: Juga memiliki kompleksitas waktu $O(n^2)$, tetapi lebih efisien daripada Selection Sort untuk daftar kecil.

3.4 Quicksort

- Definisi: Algoritma yang lebih kompleks yang menggunakan elemen pivot untuk membagi daftar menjadi dua bagian, kemudian mengurutkan kedua bagian tersebut secara rekursif.
- Sintaks:

```
def quicksort(arr):  
    if len(arr) <= 1:  
        return arr  
    pivot = arr[len(arr) // 2]  
    left = [x for x in arr if x < pivot]  
    middle = [x for x in arr if x == pivot]  
    right = [x for x in arr if x > pivot]  
    return quicksort(left) + middle + quicksort(right)
```

- Kelebihan: Memiliki kompleksitas waktu rata-rata $O(n \log n)$, menjadikannya lebih efisien untuk daftar besar.

3.5 Struktur Data Terkait

- Binary Trees: Struktur data yang menyimpan data dalam bentuk pohon, di mana setiap node memiliki maksimal dua anak.
- Heaps: Struktur data yang digunakan untuk menyimpan data dalam urutan tertentu, sering digunakan dengan algoritma pengurutan.

3.6 Time Complexity dan Space Complexity pada Algoritma Sorting

3.6.1 Selection Sort

Selection sort adalah algoritma pengurutan yang bekerja berdasarkan prinsip yang sangat sederhana. Ambil sebuah array item dan iterasi dari kiri ke kanan. Mulai dari posisi pertama pada indeks, iterasi melalui seluruh array dan tukar nilai ini dengan nilai terendah yang ditemukan di sebelah kanan item ini. Ulangi hingga seluruh array terurut.

Selection sort memiliki:

- Kompleksitas waktu terburuk adalah $O(N^2)$
- Kompleksitas waktu rata-rata adalah $O(N^2)$
- Kompleksitas waktu terbaik adalah $O(N^2)$
- Kompleksitas ruang: $O(1)$ Tambahan.

Untuk melakukan seleksi sort, ikuti langkah-langkah berikut:

- Temukan nilai terkecil dan tukar dengan nilai pertama dalam array
- Temukan nilai terkecil kedua dan tukar dengan posisi kedua dalam array
- Ulangi hingga semua item terurut dari terkecil ke terbesar

Kompleksitas waktu ditentukan berdasarkan jumlah transaksi yang dilakukan. Untuk daftar berukuran n , compiler harus mencari setiap entri dalam daftar untuk mengidentifikasi item terkecil, lalu melakukan pertukaran ke posisi indeks 0. Pseudocode algoritma tersebut adalah sebagai berikut.

```
for (i = 0; i < n - 1; i++) {
    int min_index = i;
    for (j = i + 1; j < n; j++) {
        if (List[j] < List[min_index]) {
            min_index = j;
        }
    }
    // Swap the elements
    swap(List[i], List[min_index]);
}
```

Baris 1 menyatakan bahwa panjang daftar List harus dicari sebanyak $n - 1$ kali. Baris 2 menetapkan variabel sementara untuk menyimpan nilai terendah. Baris 3 adalah loop dalam yang harus diulang sebanyak $n - 1$ kali. Baris 4 memeriksa apakah nilai yang ditemukan di posisi List[j] lebih kecil dari nilai terendah saat ini. Jika ya, posisi elemen tersebut dicatat. Di akhir setiap loop dalam, nilai yang ditemukan sebagai nilai terendah ditukar dengan posisi i dalam indeks, i ditambah satu, dan prosedur dimulai lagi. Selalu periksa item berikutnya dalam daftar hingga semua item telah diperiksa.

Ada empat hal yang perlu dipertimbangkan saat mengevaluasi algoritma ini.

1. Skenario terburuk: Jika diberikan daftar yang diurutkan dalam urutan terbalik, berapa banyak perbandingan yang dilakukan? Loop dalam dan luar harus dijalankan n kali, sehingga dapat ditentukan bahwa skenario terburuk = $O(n^2)$.
2. Perbandingan rata-rata: Terlepas dari urutan daftar, setiap item harus diperiksa, sehingga kasus rata-rata = $O(n^2)$.
3. Perbandingan terbaik: Jika diberikan daftar yang sudah diurutkan, berapa banyak perbandingan yang harus dilakukan? Lagi pula, terlepas dari item dalam daftar, setiap item harus diperiksa, sehingga kasus terbaik = $O(n^2)$.
4. Akhirnya, apa kompleksitas ruang dari pendekatan ini? Karena pertukaran di tempat dilakukan, tidak diperlukan array sementara. Ada tiga variabel sementara i, j, dan min_index; namun, variabel-variabel ini tidak bergantung pada ukuran daftar. Oleh karena itu, bayangkan menggunakan daftar, tetapi kompleksitas ruang tidak meningkat sesuai. Oleh karena itu, kompleksitas ruang = $O(1)$.

Saat mengevaluasi kompleksitas waktu, aturan praktis yang baik adalah mempertimbangkan apa yang akan terjadi jika daftar tersebut digandakan. Secara alami, loop dalam dan loop luar harus ditambah dengan jumlah iterasi yang sama untuk menyesuaikan dengan elemen tambahan dalam daftar. Oleh karena itu, dapat disimpulkan bahwa kompleksitas waktu meningkat seiring dengan ukuran daftar.

3.6.2 Quicksort

Quicksort adalah metode pengurutan yang menggunakan pendekatan divide-conquer. Diberikan sebuah array, sebuah tempat ditentukan pada array tersebut untuk membagi array mejadi dua bagian dan ini disebut pivot point. Semua nilai yang lebih besar dari titik ini dipindahkan ke kanan, dan semua nilai yang lebih kecil dari titik ini dipindahkan ke kiri. Pada langkah ini, Anda memiliki dua array. Proses yang sama diterapkan pada array-array ini hingga tidak ada elemen yang tersisa untuk diurutkan.

Quicksort memiliki:

- Kompleksitas waktu terburuk $O(n^2)$
- Kompleksitas waktu rata-rata $O(n \log n)$
- Kompleksitas waktu terbaik $O(n \log n)$
- Kompleksitas ruang $O(n)$

Untuk melakukan quicksort, ikuti langkah-langkah berikut:

- Pilih titik pada daftar sebagai pivot.
- Bagi daftar menjadi dua daftar, item di sebelah kiri pivot dan item di sebelah kanan.
- Atur variabel i untuk mengiterasi dari kiri ke kanan di sebelah kiri pivot. Atur variabel j untuk mengiterasi dari kanan ke kiri, mulai dari ujung array menuju pivot.
- Variabel i di sebelah kiri mencari nilai yang lebih besar atau sama dengan pivot. Variabel j di sebelah kanan mencari nilai yang lebih kecil atau sama dengan pivot.
- Ketika $j < i$, nilai pada posisi indeks tersebut ditukar. Proses ini diulang hingga i dan j bertemu di titik pivot.
- Bagi nilai-nilai daftar menjadi dua daftar, satu di sebelah kiri dan satu di sebelah kanan pivot. Ulangi proses ini pada masing-masing array hasil.
- Terapkan algoritma secara rekursif.

Pseudocode di bawah ini untuk quicksort dilakukan secara rekursif.

- Mulai dari elemen paling kiri, setiap elemen berikutnya diperiksa, dan jika ditemukan lebih kecil, elemen tersebut ditukar.
- Baris 4 memanggil metode partition, yang dimulai pada baris 11.
- Baris 12 menentukan elemen yang lebih besar untuk ditempatkan di sisi kanan daftar. Baris 13 menetapkan variabel i ke indeks elemen yang lebih kecil. Variabel j kemudian digunakan untuk memeriksa elemen-elemen di sebelah kanan untuk dibandingkan dengan elemen terkecil saat ini.
- Baris 12 menentukan apakah perlu pertukaran; elemen yang lebih kecil di sebelah kanan harus dipindahkan ke posisi indeks saat ini. Baris 5 digunakan untuk mengurutkan array kiri.
- Baris 6 digunakan untuk mengurutkan array kanan. Pada setiap iterasi, ukuran array yang akan diurutkan dibagi dua. Array akan terus dibagi hingga hanya tersisa satu elemen di subarray. Hasil panggilan partition akan menentukan lokasi elemen saat ini. Lokasi ini ditambah satu dan diulang hingga setiap elemen berada di posisi yang secara alami terurut.

```
// QuickSort function
void QuickSort(int List[], int low, int high) {
    if (low < high) {
        int pivot = Partition(List, low, high); // Partition
                                                the list and get pivot
        QuickSort(List, low, pivot - 1);        //
                                                Recursively sort the left subarray
        QuickSort(List, pivot + 1, high);       //
                                                Recursively sort the right subarray
    }
}

// Partition function
int Partition(int List[], int low, int high) {
    int pivot = List[high]; // Choose the last element as
                           the pivot
    int i = low - 1;        // Index for the smaller element

    for (int j = low; j <= high - 1; j++) {
        if (List[j] <= pivot) {
            i++;
            swap(List[i], List[j]); // Swap elements smaller
                                   than pivot to the left
        }
    }

    swap(List[i + 1], List[high]); // Place pivot in the
                                   correct position
    return (i + 1);                // Return the partition
                                   index
}
```

```

}

// Swap function (you can implement this as a utility
// function)
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

```

Hal-hal yang perlu dipertimbangkan saat mengevaluasi algoritma ini:

- Skenario terburuk: hal ini terjadi ketika elemen paling signifikan secara konsisten dipilih sebagai titik pivot. Hal ini akan menyebabkan loop mengiterasi setiap elemen n dari kiri. Pembagian akan menyebabkan pencarian setiap elemen di kanan tanpa elemen di kiri, $O(n^2)$.
- Skenario rata-rata: titik pivot rata-rata dipilih pada setiap panggilan. Hal ini akan mengurangi jumlah iterasi tambahan yang diperlukan. Jadi, akan ada n iterasi dan panggilan iterasi $\log n$ yang terus berkurang, $O(n * \log n)$.
- Skenario terbaik: Nilai tengah selalu dipilih, dan ruang iterasi dibagi dua pada setiap iterasi, $O(n * \log n)$.
- Sifat iteratif algoritma akan mempengaruhi kompleksitas ruang karena panggilan fungsi dan variabel disimpan di tumpukan selama perhitungan dilakukan. Namun, keputusan untuk menggunakan pertukaran di tempat berarti tidak perlu membuat array baru, $O(\log n)$.

3.6.3 Kesimpulan

Dua pendekatan penyortiran yang berbeda telah diuraikan dan dianalisis melalui perspektif kompleksitas ruang dan waktu Big-O. Telah terbukti bahwa quicksort lebih kompleks dalam implementasinya tetapi menghasilkan solusi yang lebih cepat secara keseluruhan. Selection sort lebih sederhana dan memerlukan kode yang lebih sedikit serta ruang yang lebih sedikit, tetapi tidak akan menghasilkan hasil sebaik quicksort.

3.7 Linear Searching

- Definisi: Pencarian linear adalah metode yang memeriksa setiap elemen dalam array satu per satu hingga menemukan elemen yang dicari atau mencapai akhir array.
- Sintaks:

```

for element in array:
    if element == target:
        return element

```

- Kompleksitas Waktu:
 - Best-case: $O(1)$ (elemen ditemukan di awal)
 - Worst-case: $O(n)$ (elemen tidak ada atau di akhir)

3.8 Binary Searching

- Definisi: Pencarian biner adalah metode yang memeriksa elemen tengah dari array yang sudah terurut dan membagi ruang pencarian menjadi dua bagian berdasarkan perbandingan.
- Sintaks:

```
def binary_search(array, target):
    low = 0
    high = len(array) - 1
    while low <= high:
        mid = (low + high) // 2
        if array[mid] == target:
            return mid
        elif array[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

- Kompleksitas Waktu:
 - Best-case: $O(1)$ (elemen ditemukan di tengah)
 - Worst-case: $O(\log n)$ (ruang pencarian dibagi dua setiap iterasi)

3.9 Pertimbangan dalam Searching

- Null Value: Jika nilai yang dicari tidak ada, perlu dipertimbangkan apa yang harus dikembalikan (misalnya, null).
- Pengembalian: Apakah pencarian harus mengembalikan instance pertama atau terakhir dari nilai yang dicari.

3.10 Time Complexity dan Space Complexity dalam Algoritma Searching

3.10.1 Linear Search

Pencarian linier adalah cara paling langsung untuk menemukan suatu item. Artinya, pencarian dimulai dari item pertama dan berlanjut hingga item target ditemukan atau tidak ada lagi item yang tersisa dalam array untuk diperiksa.

Jika diberikan daftar angka, mulailah dari posisi indeks 0 dan bandingkan setiap item dengan variabel target. Kembalikan hasilnya ketika posisi indeks telah ditentukan atau seluruh daftar telah diperiksa dan tidak ada instance dari elemen target.

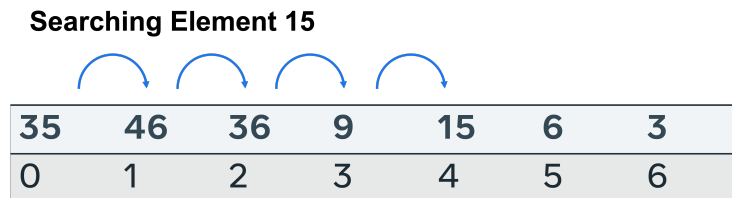


Figure 27:

Ini adalah hasil yang perlu dipertimbangkan saat mengevaluasi efektivitas pencarian.

- Kasus terburuk: Item tidak ada dalam daftar. Untuk menentukan ini, setiap lokasi mungkin dalam daftar berukuran n harus dicari. Kompleksitas waktu $O(n)$.
- Kasus rata-rata: Elemen ditemukan di tengah. Ini dianggap sebagai hasil dengan kompleksitas waktu $O(n)$.
- Kasus terbaik: Item ditemukan di indeks awal dan tidak diperlukan pemeriksaan lebih lanjut, sehingga kompleksitas waktu $O(1)$.
- Kompleksitas ruang: Tidak diperlukan ruang tambahan untuk melakukan pencarian. Oleh karena itu, ruang yang diperlukan hanya sebesar jumlah item yang harus disimpan dalam daftar, kompleksitas ruang $O(n)$.

3.10.2 Binary Search

Pencarian biner dilakukan dengan terlebih dahulu menentukan titik tengah pada daftar yang telah diurutkan, membandingkan elemen target dengan titik tengah tersebut, dan membuang setengah daftar yang nilainya lebih kecil dari elemen target. Proses pembagian setengah pada titik tengah ini diulang hingga elemen target ditemukan atau tidak ada lagi daftar yang dapat dibagi.

Untuk melakukan pencarian biner, daftar harus terlebih dahulu diurutkan.

Searching Element 6

3	6	9	15	35	36	46
0	1	2	3	4	5	6

Figure 28:

Pertama, titik tengah dipilih. Nilai pada indeks 3 adalah 15. Apakah nilai ini lebih besar atau lebih kecil dari elemen target? Ruang pencarian dibagi menjadi dua, dan bagian kiri diperiksa lebih lanjut.

Searching Element 6

3	6	9	15	35	36	46
0	1	2	3	4	5	6

Figure 29:

Titik pusat baru telah dipilih. Kali ini hanya ada tiga slot yang perlu diperiksa; lokasi indeks 1 berada di titik tengah.

Searching Element 6

3	6	9	15	35	36	46
0	1	2	3	4	5	6

Figure 30:

Nilai target ditemukan di sini dan tidak diperlukan pembagian lebih lanjut. Berikut adalah hasil yang perlu dipertimbangkan saat mengevaluasi efektivitas pencarian:

- Kasus terburuk: Item tidak ada dalam daftar. Karena sifat pendekatan ini, banyak item dihapus menggunakan operator logika lebih besar dan lebih kecil. Ini berarti hanya $n/2$ yang diperiksa pertama kali, kemudian $n/4$ dan $n/8$. Kompleksitas keseluruhan adalah $O(\log N)$.
- Kasus rata-rata: Elemen ditemukan setelah beberapa iterasi. Lagi-lagi karena mekanisme pencarian, setiap panggilan berikutnya mengurangi ruang keadaan. Jadi, dapat ditentukan bahwa setelah jumlah pencarian sedang, kompleksitasnya adalah $O(\log N)$.

- Kasus terbaik: Item ditemukan di indeks awal dan tidak diperlukan pemeriksaan lebih lanjut, sehingga $O(1)$.
- Kompleksitas ruang: Tidak diperlukan ruang tambahan untuk melakukan pencarian. Algoritma hanya menggunakan beberapa variabel (seperti target dan penghitung indeks), sehingga kompleksitas ruang adalah $O(1)$.
- Catatan: Beberapa sumber mungkin menyatakan $O(n)$ jika mereka menganggap daftar input itu sendiri sebagai ruang, tetapi secara umum kita hanya menghitung ruang tambahan yang digunakan oleh algoritma, yang adalah $O(1)$.

3.10.3 Kesimpulan

Kompleksitas waktu dan ruang untuk pencarian linier dan biner telah dianalisis. Kedua pendekatan ini merupakan pencarian in-place; oleh karena itu, tidak diperlukan ruang tambahan, sehingga kompleksitas ruangnya kecil. Pencarian linier terbukti lebih kompleks secara waktu dalam semua kasus, kecuali ketika item yang benar ditemukan pada upaya pertama.

Pencarian biner membagi ruang keadaan menjadi dua pada setiap iterasi, sehingga jauh lebih efisien. Namun, Anda harus mempertimbangkan waktu yang dibutuhkan untuk mengurutkan array dalam perhitungan keseluruhan saat mengevaluasi keefektifan pemilihan pendekatan ini untuk aplikasi umum.

3.11 Prinsip Dasar Paradigma Divide and Conquer

- Divide: Input dibagi menjadi segmen-segmen yang lebih kecil dan diproses secara individu.
- Conquer: Setiap tugas yang terkait dengan segmen yang diberikan diselesaikan.
- Combine (Opsional): Menggabungkan semua segmen yang telah diselesaikan. Langkah ini tidak selalu diperlukan.

3.11.1 Contoh: Merge Sort

Merge Sort adalah algoritma yang digunakan untuk mengurutkan array dengan membagi array menjadi dua bagian, kemudian membagi lagi hingga tersisa satu elemen, dan akhirnya menggabungkan kembali elemen yang telah diurutkan.

Sintaks:

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        L = arr[:mid]
        R = arr[mid:]
        merge_sort(L)
        merge_sort(R)
```

```

i = j = k = 0
while i < len(L) and j < len(R):
    if L[i] < R[j]:
        arr[k] = L[i]
        i += 1
    else:
        arr[k] = R[j]
        j += 1
    k += 1
while i < len(L):
    arr[k] = L[i]
    i += 1
    k += 1
while j < len(R):
    arr[k] = R[j]
    j += 1
    k += 1

```

3.12 Keuntungan Paradigma Divide and Conquer

- Parallelization: Memungkinkan beberapa thread atau komputer bekerja pada masalah yang sama secara bersamaan, mempercepat waktu penyelesaian.
- Memory Management: Mengelola memori dengan memproses data dalam potongan, terutama ketika data terlalu besar untuk dimuat dalam memori.

3.13 Apa itu Rekursi?

Rekursi adalah metode di mana sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan sub-masalah hingga kondisi keluar terpenuhi.

3.14 Tiga Persyaratan untuk Solusi Rekursif

- Base Case: Kondisi yang memastikan fungsi tidak terus memanggil dirinya sendiri. Contoh: Jika n sama dengan 0, fungsi akan berhenti.
- Diminishing Structure: Memastikan bahwa input berkurang setiap kali fungsi dipanggil, sehingga akhirnya mencapai base case.
- Recursive Call: Memanggil fungsi itu sendiri dengan input yang lebih kecil. Contoh sintaks:

```

def exponent(x, n):
    if n == 0:
        return 1 # Base case
    else:
        return x * exponent(x, n - 1) # Recursive call

```

3.15 Contoh Penggunaan Rekursi

- Binary Search: Fungsi yang menerima daftar dan nilai target, memeriksa titik tengah daftar, dan memanggil dirinya sendiri hingga elemen target ditemukan atau tidak ada.
- Fibonacci: Menghitung angka Fibonacci dengan memanggil fungsi secara rekursif untuk menjumlahkan dua angka sebelumnya.

3.16 Keuntungan Menggunakan Rekursi

- Keterbacaan: Kode rekursif sering kali lebih mudah dibaca dibandingkan dengan loop yang kompleks.
- Divide and Conquer: Rekursi sering digunakan dalam pendekatan ini, membagi masalah menjadi langkah-langkah yang lebih kecil untuk menemukan solusi optimal.

3.17 Dynamic Programming

- Definisi: Dynamic programming adalah paradigma pemrograman yang memecahkan masalah dengan membaginya menjadi sub-masalah yang lebih kecil dan menyimpan solusi untuk digunakan kembali.
- Keuntungan: Menghindari perhitungan ulang sub-masalah yang sama dengan mencari solusi yang sudah disimpan.

3.18 Memoization

- Definisi: Memorization adalah teknik menyimpan hasil perhitungan dari sub-masalah untuk menghindari perhitungan ulang di masa depan.
- Contoh: Jika Anda menghitung 2^3 , hasilnya disimpan dan digunakan kembali saat menghitung 2^6 .

3.19 Recursion

- Definisi: Recursion adalah metode pemrograman di mana fungsi memanggil dirinya sendiri untuk menyelesaikan sub-masalah.
- Contoh: Fungsi yang menghitung nilai Fibonacci dapat menggunakan recursion untuk memanggil dirinya sendiri.

3.20 Contoh Masalah terkait Dynamic Programming

- Fibonacci Sequence: Contoh masalah kombinasi yang sering digunakan dalam wawancara kerja.

- Knapsack Problem: Masalah optimasi di mana Anda harus memilih kombinasi item dengan berat dan nilai tertentu untuk memaksimalkan nilai dalam kapasitas tertentu.

3.21 Langkah-langkah dalam Dynamic Programming

- Tentukan Objective Function: Deskripsikan hasil optimal yang ingin dicapai.
- Pecah Masalah: Bagi masalah menjadi langkah-langkah yang lebih kecil.
- Gunakan Recursive Functions: Tulis fungsi yang dapat memanggil dirinya sendiri untuk menyelesaikan masalah.

3.21.1 Sintaks Contoh

Contoh Sintaks untuk Fibonacci:

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
```

3.22 Prinsip Dasar Greedy Algorithm

- Occam's Razor: Prinsip filosofis yang menyatakan bahwa solusi yang paling sederhana biasanya adalah yang terbaik. Dalam konteks ini, algoritma greedy dianggap sebagai solusi yang sederhana.
- Greedy Algorithm: Pendekatan yang mencari solusi lokal yang terbaik pada setiap langkah, tanpa mempertimbangkan solusi global.

3.23 Perbandingan Greedy Algorithm dengan Dynamic Programming

- Dynamic Programming: Mencari solusi optimal secara global dengan membagi masalah menjadi sub-masalah dan menyelesaikannya secara menyeluruh.
- Greedy Approach: Memilih opsi yang paling menguntungkan saat ini tanpa mempertimbangkan konsekuensi di masa depan.

3.24 Contoh Penerapan Greedy Algorithm dan Dynamic Programming

CPU Scheduling:

- Dynamic Programming: Memilih subset aktivitas yang dapat diselesaikan dalam waktu tertentu.
- Greedy Approach: Memilih program dengan waktu eksekusi terpendek terlebih dahulu.

Knapsack Problem:

- Dynamic Programming: Menentukan subset item yang memaksimalkan nilai.
- Greedy Approach: Memilih item dengan nilai tertinggi tanpa memper-timbangkan item lain yang mungkin lebih menguntungkan.

3.25 Contoh Jalur Terpendek

- Dynamic Programming: Menghitung biaya dari setiap node untuk mene-mukan jalur terpendek secara menyeluruh.
- Greedy Approach: Memilih node dengan biaya terendah secara langsung tanpa menghitung semua kemungkinan.

3.26 Kesimpulan terkait Greedy Algorithm

Algoritma greedy menawarkan solusi yang cepat dan mudah diimplementasikan, tetapi tidak selalu menjamin hasil yang optimal. Memahami perbedaan antara kedua pendekatan ini penting untuk memilih metode yang tepat dalam menye-lesaikan masalah.